



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

08-任务映射与芯片工具链

贾弘洋 & 刘勇攀

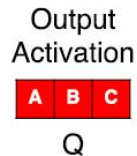
2025/4/8



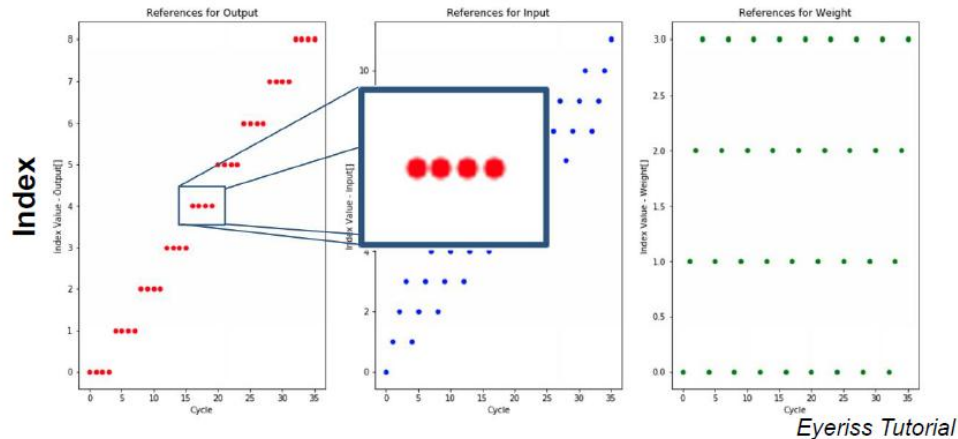
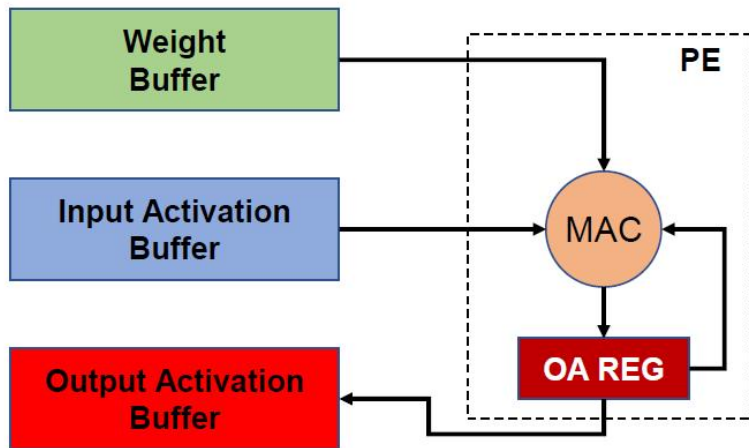


- Core PE design and data reuse: **MAC, OS/WS**
- Inefficiency in CPU: **IF, D-\$ (data access)**
- Core optimization
 - Instruction decoding: **coarse-grained, domain-specific, operation fusion**
 - Datapath: **spatial-K (accumulation), spatial-M/N (data reuse)**
 - Memory system: **short-lived values, dedicated storage, data delivery**
- CASE study
 - Google TPU: **systolic architecture**
 - Huawei Ascend (昇腾): **SIMD architecture**

/Data Reuse with Buffer (OS)

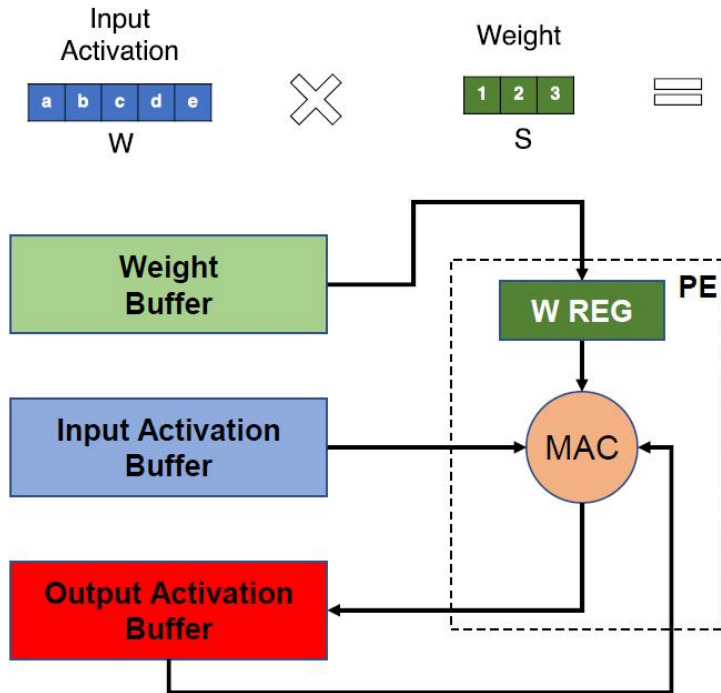


```
for (q=0; q<Q; q++){ // Q =9
    for (s=0; s<S; s++){ // S=4
        OA[q] += IA[q+s] * W[s];
    }
}
```



Observation: Single **output** is reused S times.

/Data Reuse with Buffer (WS)

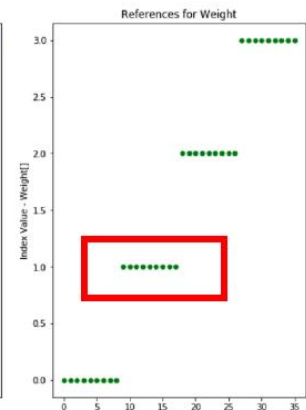
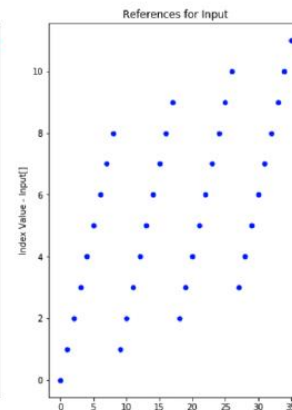
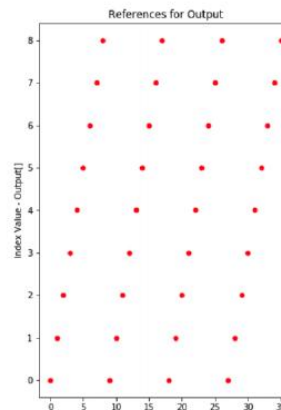


Output Activation

A	B	C
---	---	---

Q

```
for (s=0; s<S; s++) { // S=4
  for (q=0; q<Q; q++) { // Q =9
    OA[q] += IA[q+s] * W[s];
  }
}
```



Eyeriss Tutorial

Observation: Single **weight** is reused Q times.

/Source of Inefficiency in CPUs



```

10000170: 000c0993      + addi x19,x24,-256
10000174: 00004a97      + auipc x21,0x4
10000178: 674a0a93      + addi x21,x21,1652 # 100047e8 <B>
1000017c: 000a8913      + mv x18,x21
10000180: 000a8413      + mv x8,x23
10000184: 00000493      + li x9,0
10000188: 00092583      + lw x11,0(x18)
1000018c: 00042583      + lw x10,0(x8)
10000190: 00440413      + addi x8,x8,4
10000194: 10090913      + addi x18,x18,256
10000198: f25ff0ef      + jal x1,100000bc <times>
1000019c: 00a484b3      + add x9,x9,x10
100001a0: 0099a023      + beq x9,0(x10)
100001a4: f14412e3      + bne x8,x20,10000188 <mult+0x6c>
100001a8: 00498993      + addi x19,x19,4
100001ac: 0099a033      + add x22,x22,x9
100001b0: 004a8a93      + addi x21,x21,4
100001b4: fd8994e3      + bne x19,x24,1000017c <mult+0x60>
100001b8: 10040a13      + addi x20,x8,256
100001bc: 10098c13      + addi x24,x19,256
100001c0: 100b8b93      + addi x23,x23,256
100001c4: 7b9a16e3      + bne x20,x25,10000170 <mult+0x54>
    
```

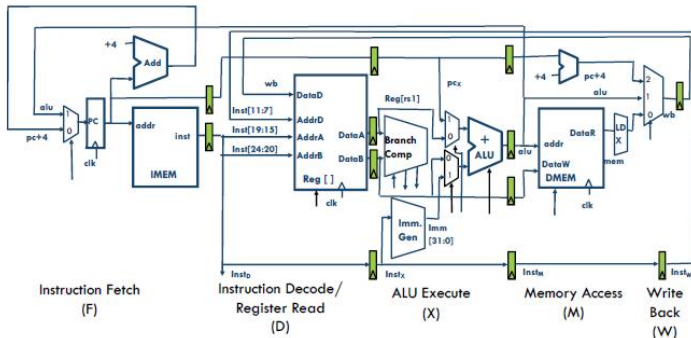
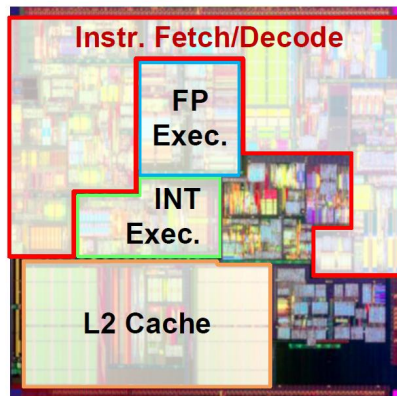


Table 3. Datapath energy breakdown for our base implementation in mJ/frame. IF is instruction fetch/decode (including the I-cache). D-\$ is the D-cache. Pip is the pipeline registers, buses, and clocking. Ctl is random control. RF is the register file. FU is the functional elements. Data estimates from processor simulations.

	IF	D-\$	Pip	Ctl	RF	FU	Total
IME	410	218	257	113	113	68	1179
FME	286	196	205	90	90	54	921
Intra	54	20	29	13	13	8	137
CABAC	12	2	8	4	4	2	32
Total	762	436	499	220	220	132	2269

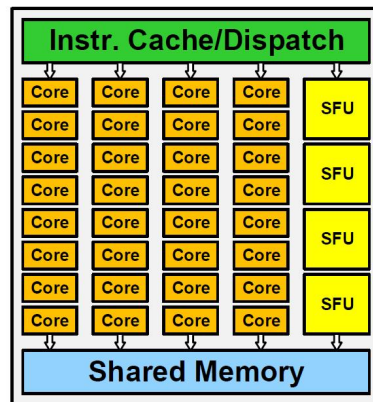
/Source of Inefficiency in CPUs

Intel Pentium 4 CPU



130nm CMOS, $\sim 140\text{mm}^2$
Area for Prog. $\sim 40\%$

Nvidia Fermi SM



32 CUDA Cores
4 Special Function Units (SFU)

- Instruction Fetching and Decoding occupy large silicon area for CPUs.
- GPU use more cores (SIMD) to amortize instruction-related overheads.

/ Inst. Decoding Logic Optimization

- Coarse-Grained, Domain-Specific Instructions
 - Key instructions: data movement and compute
- Example: Gemmini ISAs

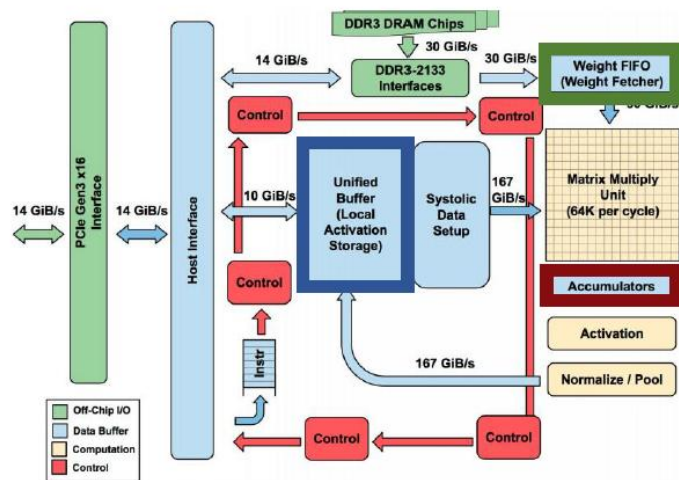
```
10000170: f00c0993      + addi+ x19,x24,-250
10000174: 00004a97      + auipc+ x21,0x4
10000178: 674a8a93      + addi+ x21,x21,1652 # 100047e8 <8>
1000017c: 000a8913      + mv+ x18,x21
10000180: 000a8413      + mv+ x8,x23
10000184: 00000493      + li+ x9,0
10000188: 00092583      + lw+ x11,0(x18)
1000018c: 00042503      + lw+ x10,0(x8)
10000190: 00440413      + addi+ x8,x8,4
10000194: 10090913      + addi+ x18,x18,256
10000198: f25ff0ef      + jal+ x1,1000000c <times>
1000019c: 00a484b3      + add+ x9,x9,x10
100001a0: 0099a023      + sw+ x9,0(x19)
100001a4: ff4412e3      + bne+ x8,x8,10000188 <mult+0x6c>
100001a8: 00498993      + addi+ x19,x19,4
100001ac: 009b0b33      + add+ x22,x22,x9
100001b0: 004a8a93      + addi+ x21,x21,4
100001b4: fd8994e3      + bne+ x19,x24,1000017c <mult+0x60>
100001b8: 10040a13      + addi+ x20,x8,256
100001bc: 10098c13      + addi+ x24,x19,256
100001c0: 100b8b93      + addi+ x23,x23,256
100001c4: fb9a16e3      + bne+ x20,x25,10000170 <mult+0x54>
```

- Data movement:
 - **mvin**: move data from L2/DRAM to scratchpad
 - **mvout**: move data from scratchpad to L2/DRAM
- Compute:
 - **matmul.preload**: preload weights for weight stationary
 - **matmul.compute.preloaded**: compute using preloaded values
 - **matmul.compute.accumulated**: compute using existing values
- Configurations:
 - **config_ex**: configures the execute pipeline
 - **config_mvin**: configures the load pipeline
 - **config_mout**: configures the store pipeline
 - **flush**: flush the TLB

<https://github.com/ucb-bar/gemmini>



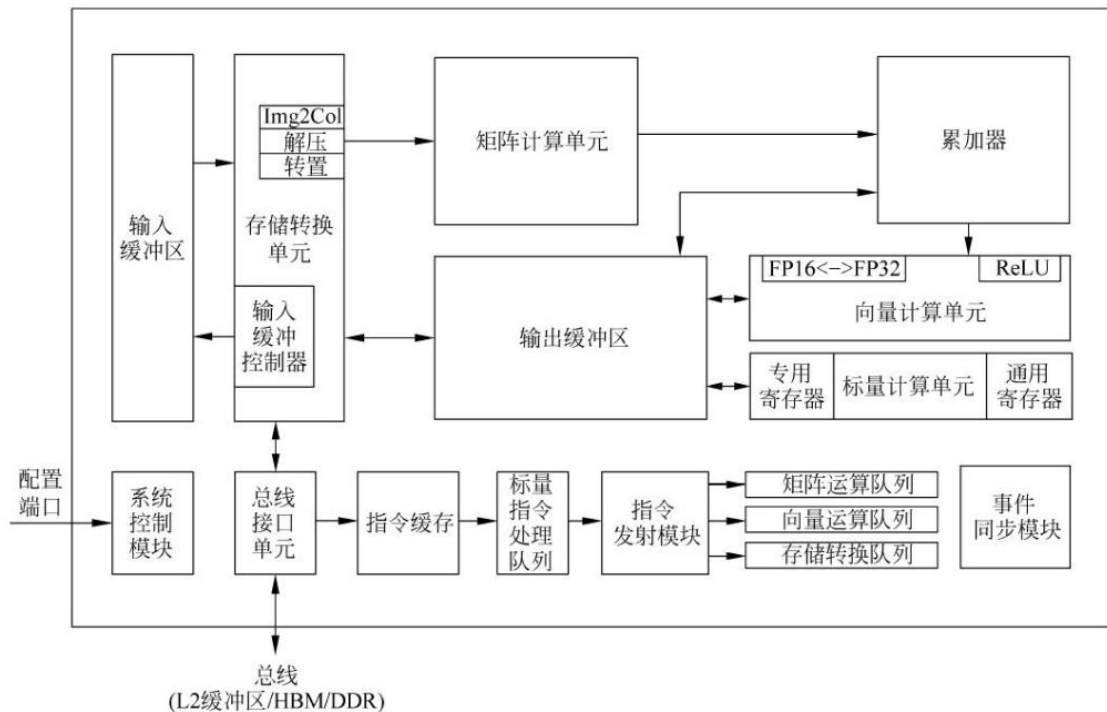
/Tensor Processing Unit



TPU

- Inst. Decoding Logic:
 - Coarse-grained matrix multiply and data read/write instructions
- Datapath:
 - Spatial-K: Systolic Accumulation
 - Multi-cycle with registers
 - Spatial-N: Systolic multicast
 - Multi-cycle with registers
 - Better scalability
- Memory
 - Custom systolic registers
 - Dedicated accumulation and weight buffers
 - Double-buffered, weight-stationary dataflow

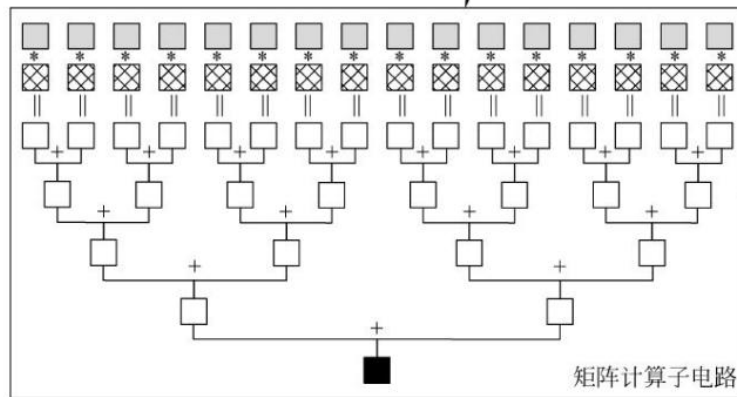
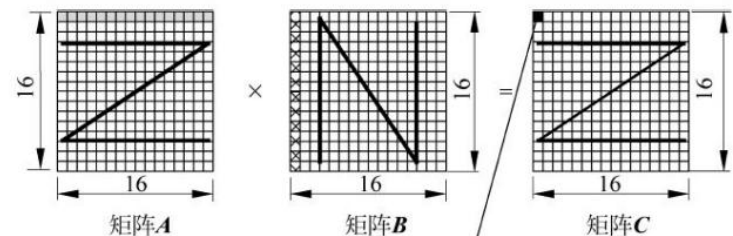
/Huawei Ascend AI Core



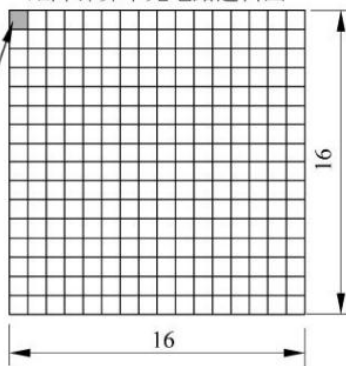
三种计算资源：矩阵计算单元，向量计算单元，标量计算单元，三条独立流水线设计，对应三个指令发射序列。

存储转换单元：硬件完成Img2Col将输入网络和特征值重排进行输入。对比GPU使用软件实现效率更高。

矩阵计算单元



矩阵计算单元电路逻辑图



采用**Adder tree**模式设计。
矩阵A：以行为顺序存储。
矩阵B：以列为顺序存储。

一个周期执行**16x16**次乘法
计算出一个值并存储。

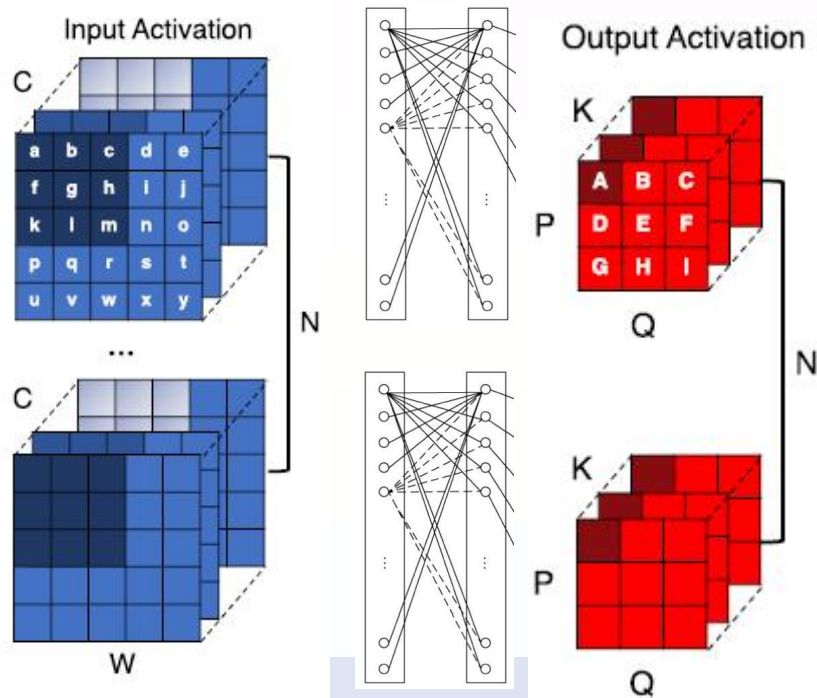
/Outlines

- **DNN Mapping Problem**
 - **Loop Nest**
 - **HW Constraints**
- **Mapping Space**
 - **Loop Ordering**
 - **Loop Bound**
 - **Spatial Choice**
- **Tuning**
- **Toolchain**

/Dataflow Recap

- Defines the execution order of the DNN operations in hardware.
 - Computation order
 - Data movement order
- Loop nest is a compact way to describe the execution order, i.e., dataflow, supported in hardware.
 - **for**: temporal for, describes the temporal execution order.
 - **spatial_for**: describes parallel execution.

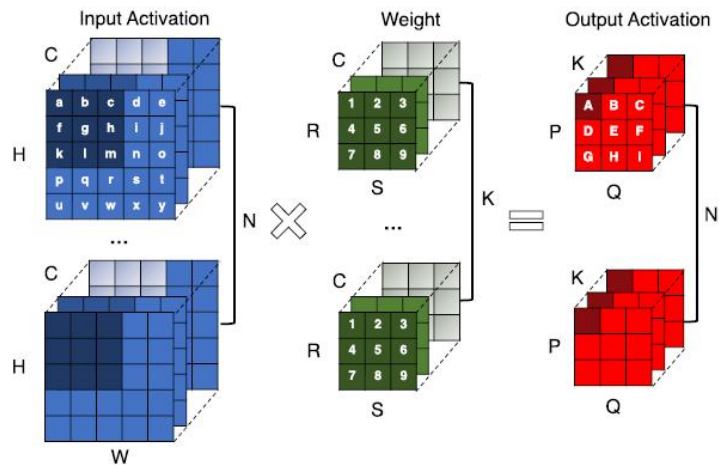
/ Fully-Connected Layer Loop Nesting



Too large calculation overhead for raw 2D image data. Now only classifier layer (vector to vector) uses FC.

```
for (n=0; n<N; n++) {  
  for (k=0; k<K; k++) {  
    for (p=0; p<P; p++) {  
      for (q=0; q<Q; q++) {  
        OA[n][k][p][q] = 0;  
        for (c=0; c<C; c++) {  
          for (h=0; h<H; h++) {  
            for (w=0; w<W; w++) {  
              OA[n][k][p][q] +=  
                IA[n][c][h][w] * W[k][p][q][c][h][w];  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

/ Convolution Layer Loop Nesting



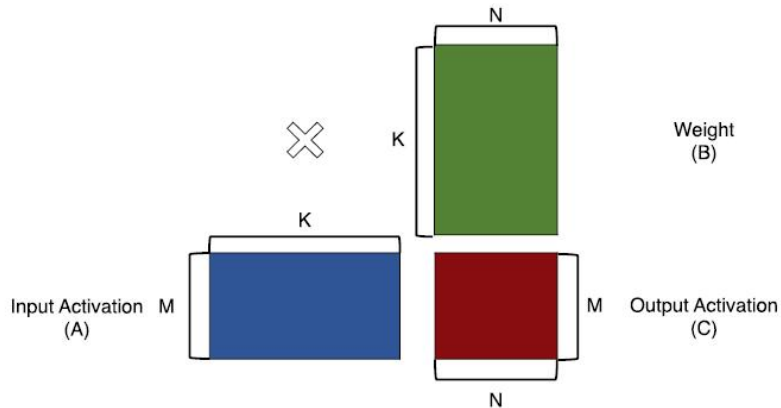
```

for (n=0; n<N; n++) {
    for (k=0; k<K; k++) {
        for (p=0; p<P; p++) {
            for (q=0; q<Q; q++) {
                OA[n][k][p][q] = 0;
                for (r=0; r<R; r++) {
                    for (s=0; s<S; s++) {
                        for (c=0; c<C; c++) {
                            h = p * stride - pad + r;
                            w = q * stride - pad + s;
                            OA[n][k][p][q] +=
                                IA[n][c][h][w]
                                * W[k][c][r][s];
                        }
                    }
                }
            }
        }
    }
    OA[n][k][p][q] = Activation(OA[n][k][p][q]);
}
    
```

for each output activation

convolution window

/Matrix Multiply

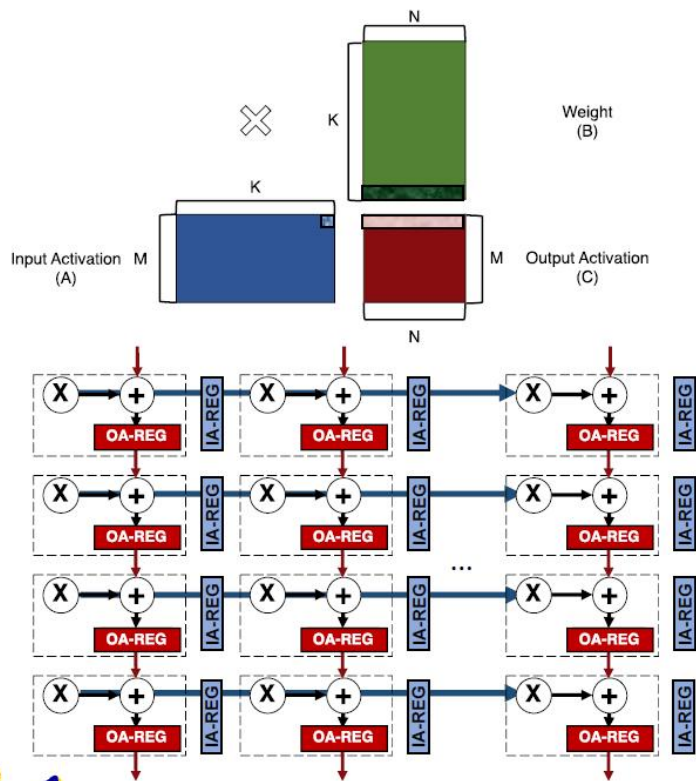


```
for (m=0; m<M; m++) {  
    for (n=0; n<N; n++) {  
        OA[n,m] = 0;  
        for (k=0; k<K; k++) {  
            OA[n,m] += IA[m, k] * W[k, n];  
        }  
        OA[n,m] = Activation(OA[n,m]);  
    }  
}
```

for each output activation

convolution window

/ Datapath Optimization Combined: TPU



```
for (m=0; m<M; m++) {  
  spatial_for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    spatial_for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
        * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

- Systolic accumulation
- Systolic multicast

/Outlines

- **DNN Mapping Problem**

- **Loop Nest**

- **HW Constraints**

- **Mapping Space**

- **Loop Ordering**

- **Loop Bound**

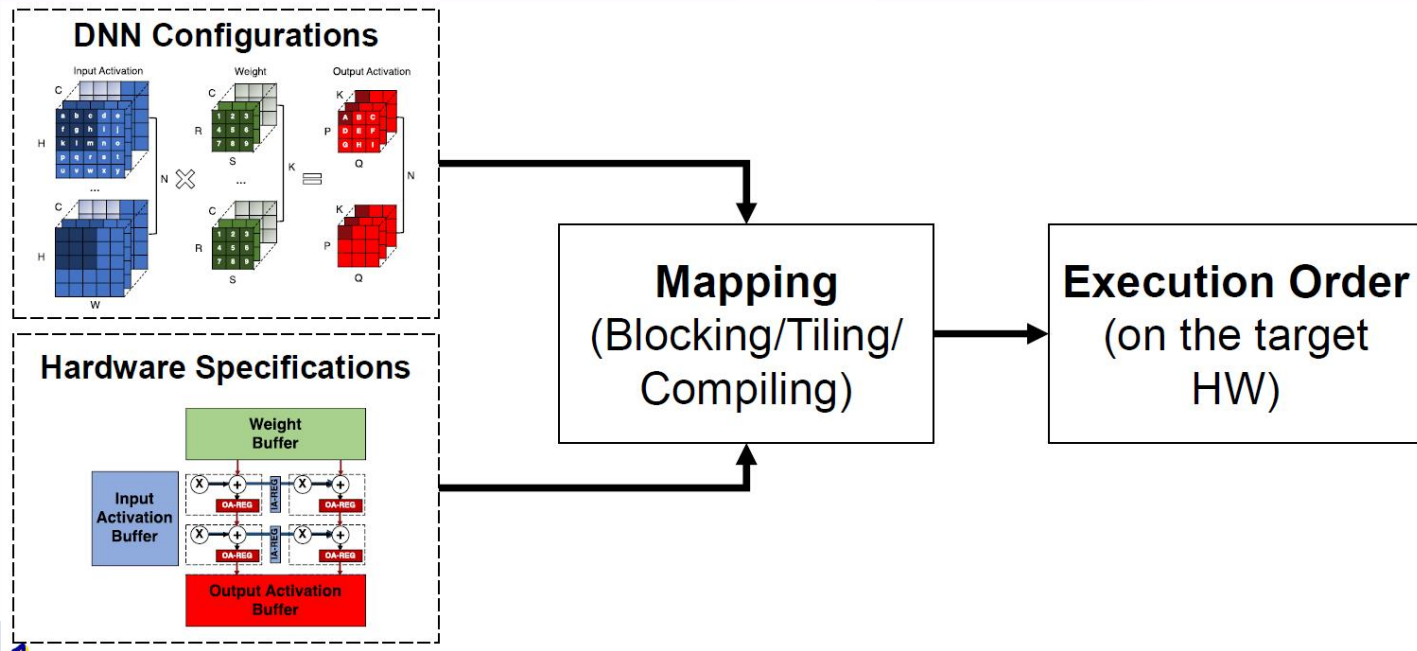
- **Spatial Choice**

- **Tuning**

- **Toolchain**

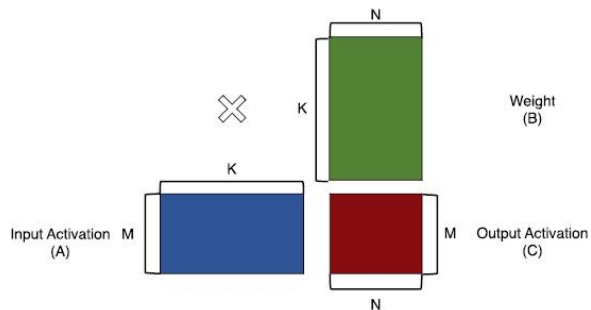
/The DNN Mapping Problem

Mapping: Large network model $\rightarrow$ limited hardware resources.
Reducing additional data movement/recompute overheads.



/Q1: What if the systolic array size $< K$ or N ?

Can not mapping the all weights to the systolic array in the same time.



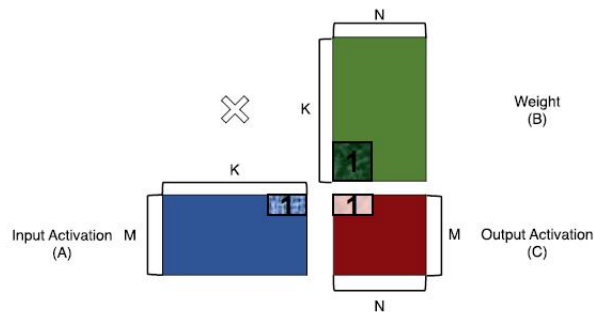
- Hardware Specifications:

- Systolic array size: 2x2

```
for (m=0; m<M; m++) {  
    spatial_for (n=0; n<2; n++) {  
        OA[n,m] = 0;  
        spatial_for (k=0; k<2; k++) {  
            OA[n,m] += IA[m, k]  
                      * W[k, n];  
        }  
        //OA[n,m] is not done yet.  
    }  
}
```

/Q1: What if the systolic array size $< K$ or N ?

Adding two bounds, i.e., $K1$ and $N1$, to mapping part of the weights.

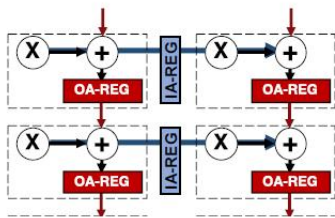


- Hardware Specifications:

- Systolic array size: 2×2

- Notation:

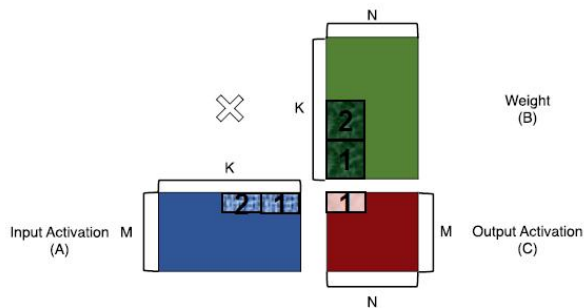
- $K?/N?$: loop bounds



```
for (m=0; m<M; m++) {  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
}
```

/Q1: What if the systolic array size $< K$ or N ?

Adding two bounds, i.e., $K1$ and $N1$, to mapping part of the weights.

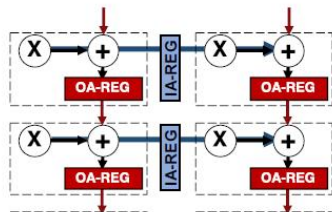


- Hardware Specifications:

- Systolic array size: 2x2

- Notation:

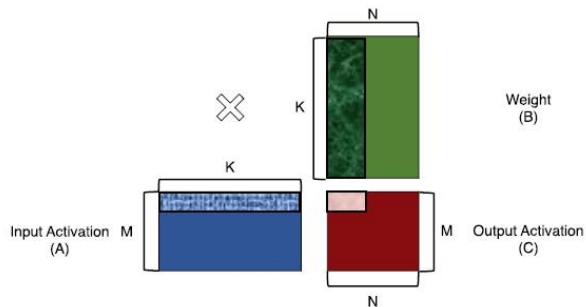
- $K1/N1$: loop bounds



```
for (m=0; m<M; m++) {  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
}
```


/Q1: What if the systolic array size $< K$ or N ?

Adding two bounds, i.e., $K1$ and $N1$, to mapping part of the weights.

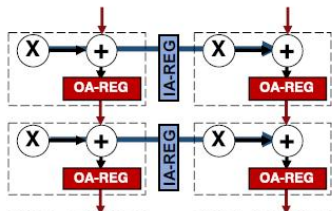


- Hardware Specifications:

- Systolic array size: 2×2

- Notation:

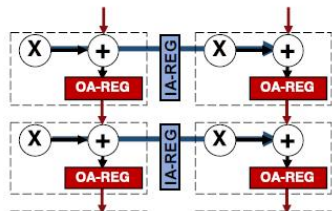
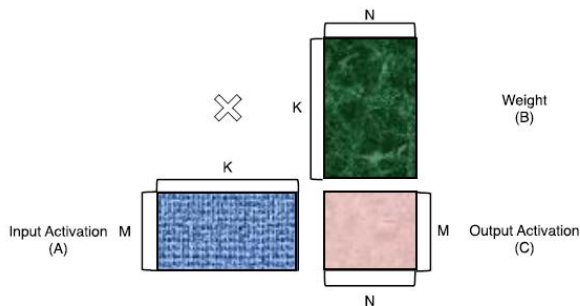
- $K1/N1$: loop bounds



```
for (m=0; m<M; m++) {  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
}
```

/Q1: What if the systolic array size $< K$ or N ?

Adding two bounds, i.e., $K1$ and $N1$, to mapping part of the weights.

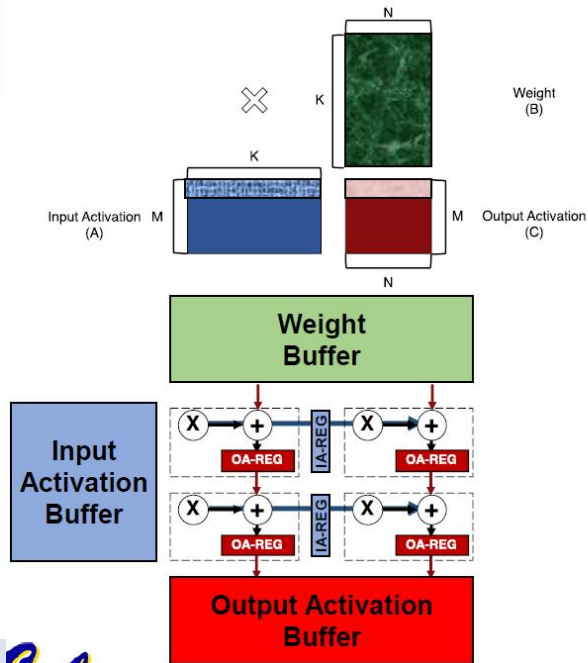


- Hardware Specifications:
 - Systolic array size: 2×2
- Notation:
 - $K1/N1$: loop bounds

```
for (m=0; m<M; m++) {  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
}
```

Q2: What if weight buffer size $< N \times K$?

Weights need to be move in/out to the weight buffer. Optimize to minimize data movements.



- Hardware Specifications:
 - Systolic array size: 2x2
 - Explicit data movement

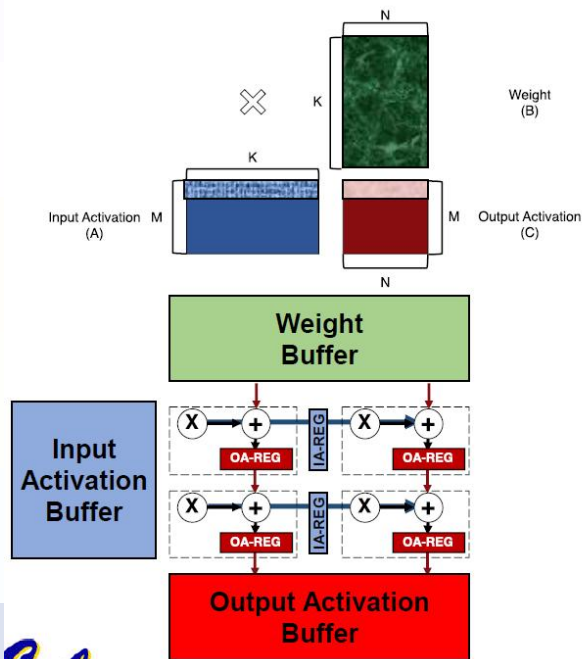
```

for (m=0; m<M; m++) {
  for (n1=0; n1<N1; n1++) {
    OA[n1*N0:(n1+1)*N0,m] = 0;
    for (k1=0; k1<K1; k1++) {
      spatial_for (n0=0; n0<N0; n0++) {
        spatial_for (k0=0; k0<K0; k0++) {
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                           * W[k1*K0+k0, n1*N0+n0];
        }
      }
    }
  }
}

```

/Q2: What if weight buffer size $< N \times K$?

Weights need to be move in/out to the weight buffer. Optimize to minimize data movements.



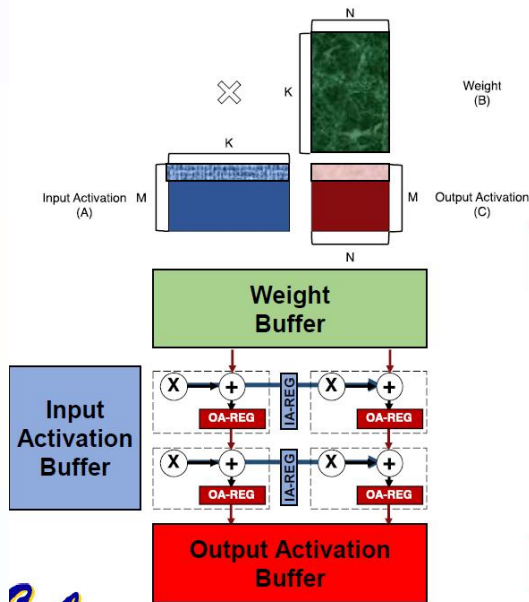
• Hardware Specifications:

- Systolic array size: 2×2
- Explicit data movement

```
for (m=0; m<M; m++) {  
  mvin(W[0:K,0:N]);  
  mvin(IA[m:m+1,0:K]);  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
  mvout(OA[0:N,m:m+1]);  
}
```

/Q2: What if weight buffer size $< N \times K$?

Weights need to be move in/out to the weight buffer. Optimize to minimize data movements.

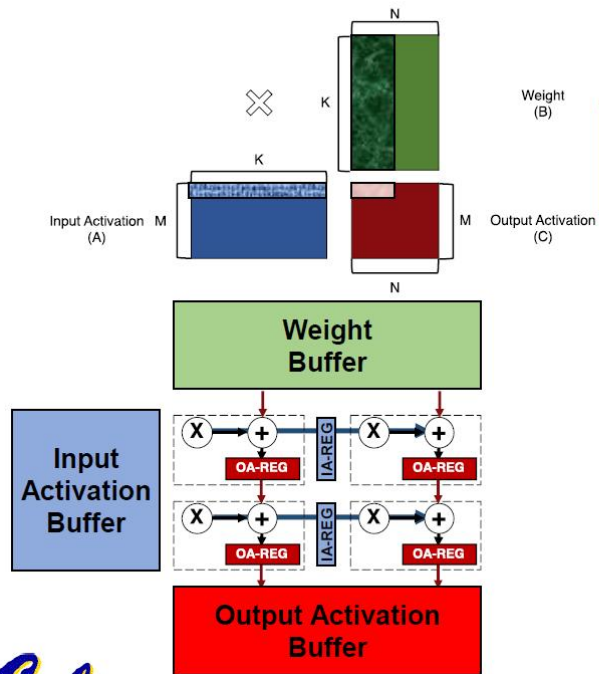


- Hardware Specifications:
 - Systolic array size: 2×2
 - Explicit data movement
 - Output/Weight/Input buffer sizes

```
for (m=0; m<M; m++) {  
  mvin(W[0:K,0:N]); // W buffer  $\geq N \times K$   
  mvin(IA[m:m+1,0:K]); // IA buffer  $\geq 1 \times K$   
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
  mvout(OA[0:N,m:m+1]); // OA buffer  $\geq 1 \times N$   
}
```

/Q2: What if weight buffer size $< N \times K$?

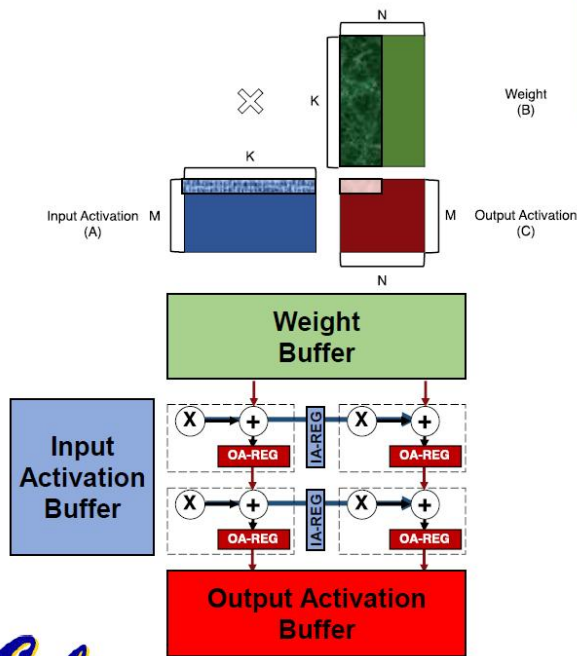
Weights need to be move in/out to the weight buffer. Optimize to minimize data movements.



```
for (m=0; m<M; m++) {  
    mvin(IA[m:m+1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K < NK$   
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);  
        OA[n2*N1*N0:(n2+1)*N1*N0, m:m+1]=0;  
        for (n1=0; n1<N1; n1++) {  
            for (k1=0; k1<K1; k1++) {  
                spatial_for (n0=0; n0<N0; n0++) {  
                    spatial_for (k0=0; k0<K0; k0++) {  
                        OA[n2*N1*N0+n1*N0+n0, m]  
                            +=IA[m, k1*K1+k0]  
                            * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];  
                    }  
                }  
            }  
        }  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0, m:m+1]);  
    }  
}
```


/Q3: What if input buffer size $> 1 \times K$?

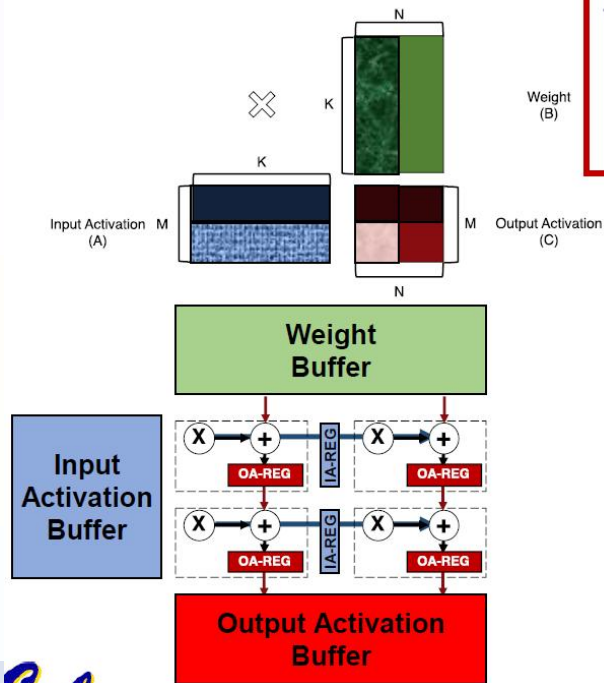
Should store > 1 rows of input activations.



```
for (m=0; m<M; m++) {  
    // IA buffer stores:  $1 \times K$   
    mvin(IA[m:m+1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K$   
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);  
        OA[n2*N1*N0:(n2+1)*N1*N0, m:m+1]=0;  
        for (n1=0; n1<N1; n1++) {  
            for (k1=0; k1<K1; k1++) {  
                spatial_for (n0=0; n0<N0; n0++) {  
                    spatial_for (k0=0; k0<K0; k0++) {  
                        OA[n2*N1*N0+n1*N0+n0, m]  
                            +=IA[m, k1*K1+k0]  
                            * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];  
                    }  
                }  
            }  
        }  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0, m:m+1]);  
    }  
}
```


/Q3: What if input buffer size $> 1 \times K$?

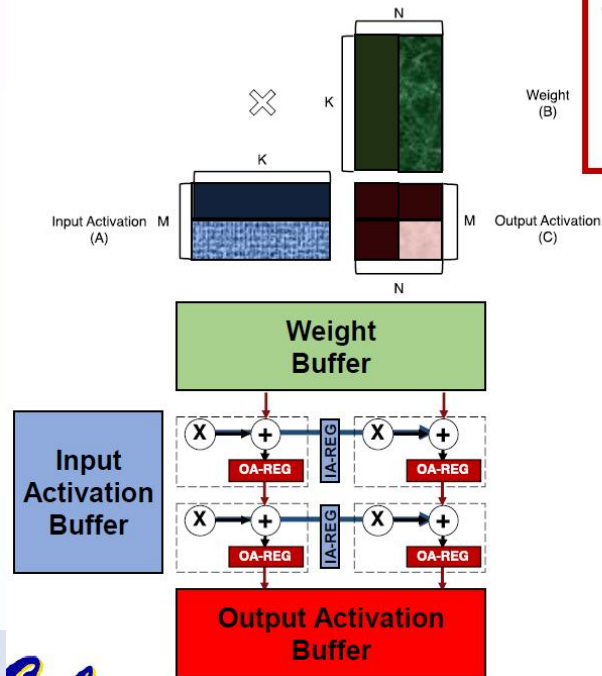
Should store > 1 rows of input activations.



```
for (m2=0; m2<M2; m2++) {  
    // IA buffer stores:  $M1 \times K$   
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K$   
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);  
        OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1]=0;  
        for (m1=0; m1<M1; m1++) {  
            for (n1=0; n1<N1; n1++) {  
                for (k1=0; k1<K1; k1++) {  
                    spatial_for (n0=0; n0<N0; n0++) {  
                        spatial_for (k0=0; k0<K0; k0++) {  
                            OA[n2*N1*N0+n1*N0+n0, m2*M1+m1]  
                                += IA[m2*M1+m1, k1*K1+k0]  
                                    * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];  
                        }  
                    }  
                }  
            }  
        }  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,  
            m2*M1:(m2+1)*M1]);  
    }  
}
```

/Q3: What if input buffer size $> 1 \times K$?

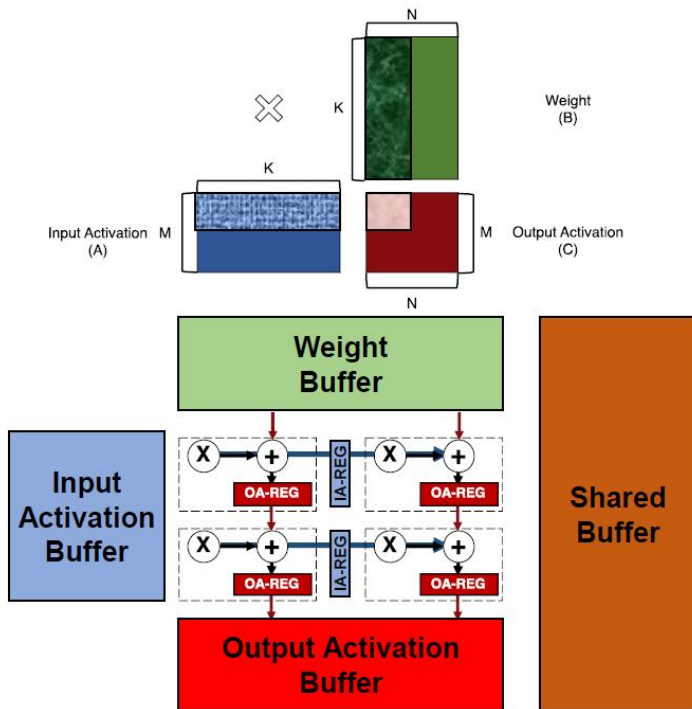
Should store > 1 rows of input activations.



```
for (m2=0; m2<M2; m2++) {  
    // IA buffer stores:  $M1 \times K$   
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K$   
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);  
        OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1]=0;  
        for (m1=0; m1<M1; m1++) {  
            for (n1=0; n1<N1; n1++) {  
                for (k1=0; k1<K1; k1++) {  
                    spatial_for (n0=0; n0<N0; n0++) {  
                        spatial_for (k0=0; k0<K0; k0++) {  
                            OA[n2*N1*N0+n1*N0+n0, m2*M1+m1]  
                                +=IA[m2*M1+m1, k1*K1+k0]  
                                * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];  
                        }  
                    }  
                }  
            }  
        }  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0  
            , m2*M1:(m2+1)*M1]);  
    }  
}
```

/Q4: What if there is another shared buffer?

Buffer for activation/weight... Depending on the problem.

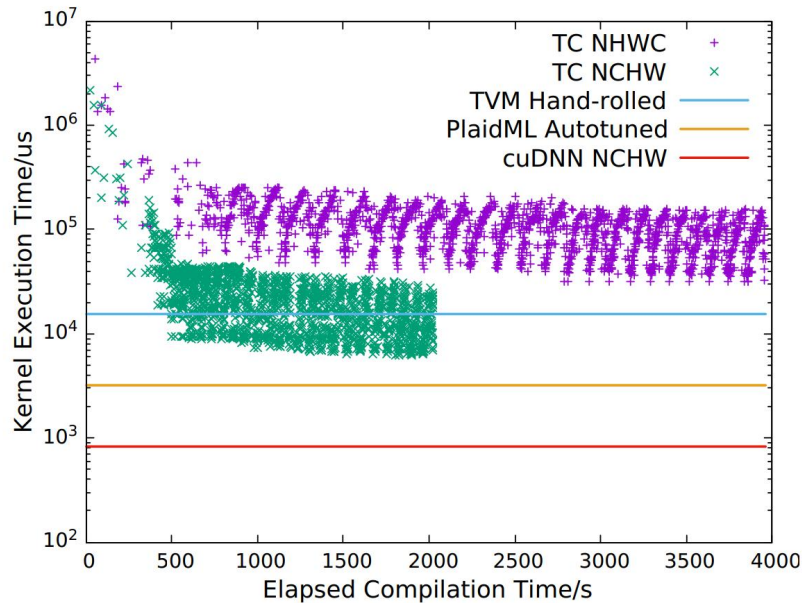


```
for (m3=0; m3<M3; m3++) {  
  for (n3=0; n3<N3; n3++) {  
    // Shared buffer blocking
```

```
  for (m2=0; m2<M2; m2++) {  
    // IA buffer stores: M1*K  
    mvin(IA[...:...]);  
    for (n2=0; n2<N2; n2++) {  
      // W buffer stores: N1*N0*K  
      mvin(W[...:...]);  
      OA[...:...]=0;  
      for (m1=0; m1<M1; m1++) {  
        for (n1=0; n1<N1; n1++) {  
          for (k1=0; k1<K1; k1++) {  
            ...  
          }  
        }  
      }  
      mvout(OA[...:...]);  
    }  
  }  
}
```

/The DNN Mapping Problem

Buffer for activation/weight... Depending on the problem.



(a) Strided conv2d (batch=32)

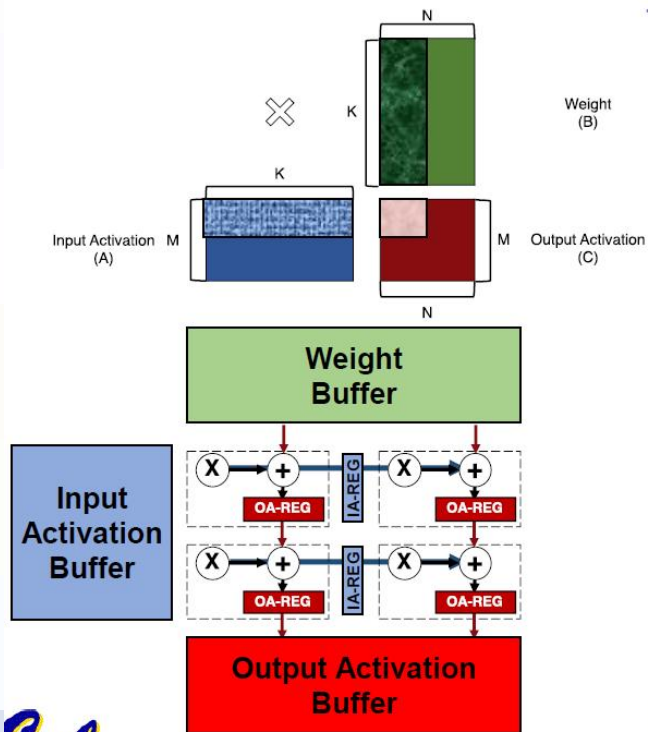
/Outlines

- **DNN Mapping Problem**
 - **Loop Nest**
 - **HW Constraints**
- **Mapping Space**
 - **Loop Ordering**
 - **Loop Bound**
 - **Spatial Choice**
- **Tuning**
- **Toolchain**

/ Mapping Dimensions

- Loop ordering:
 - Which index goes to the inner/outer loop?
- Loop bounds:
 - What are the loop bounds (i.e., $N?/K?/M?$) for each loop?
- Spatial choice:
 - Which loop should be spatial/temporal?
 - Data/Model Parallelism

/The DNN Mapping Problem

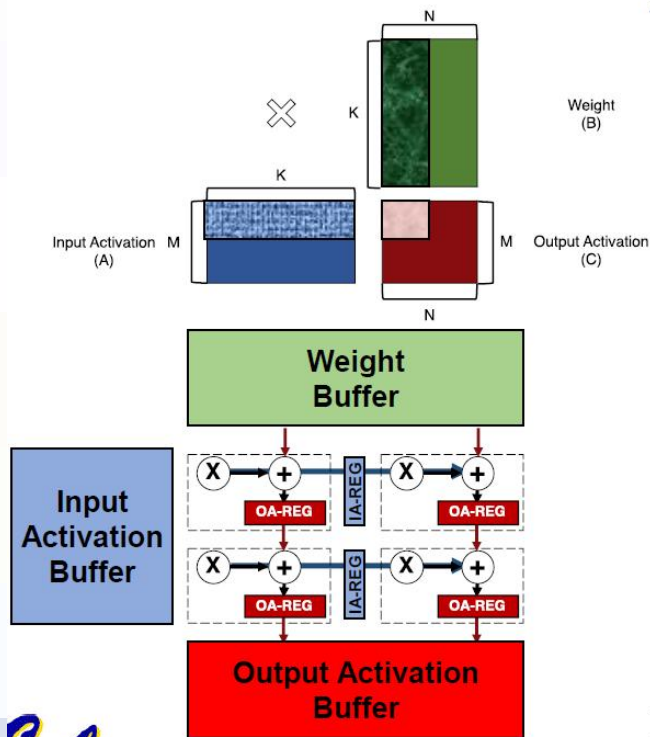


```

for (m2=0; m2<M2; m2++) {
    // IA buffer stores: M1*K
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1] = 0;
        for (m1=0; m1<M1; m1++) {
            for (n1=0; n1<N1; n1++) {
                for (k1=0; k1<K1; k1++) {
                    spatial_for (n0=0; n0<N0; n0++) {
                        spatial_for (k0=0; k0<K0; k0++) {
                            OA[n2*N1*N0+n1*N0+n0, m2*M1+m1]
                                += IA[m2*M1+m1, k1*K1+k0]
                                   * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];
                        }
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1]);
    }
}

```


/ Loop Bounds

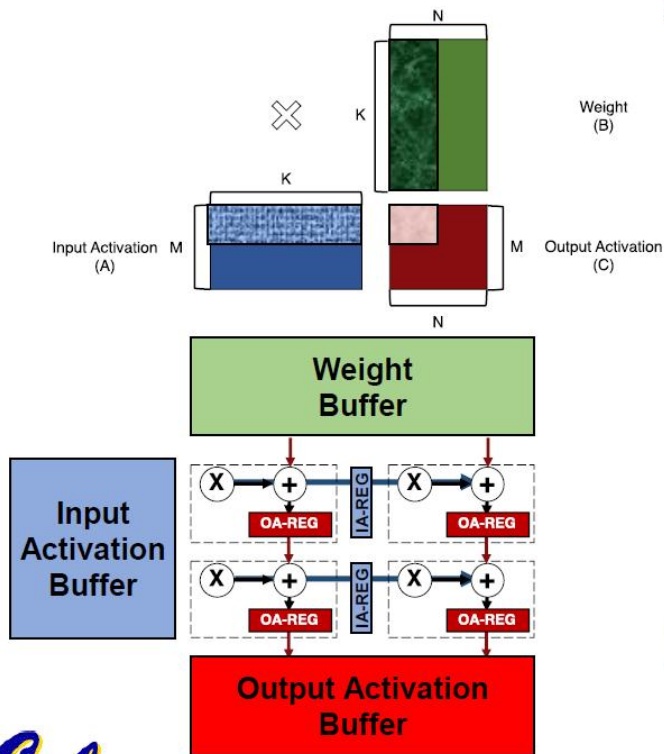


```

for (m2=0; m2<M2; m2++) {
    // IA buffer stores: M1*K
    mvn(IA[m2*M1:(m2+1)*M1, 0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvn(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1]=0;
        for (m1=0; m1<M1; m1++) {
            for (n1=0; n1<N1; n1++) {
                for (k1=0; k1<K1; k1++) {
                    spatial_for (n0=0; n0<N0; n0++) {
                        spatial_for (k0=0; k0<K0; k0++) {
                            OA[n2*N1*N0+n1*N0+n0, m2*M1+m1]
                                +=IA[m2*M1+m1, k1*K1+k0]
                                    * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];
                        }
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1]);
    }
}
    
```

$M1 * M2 = M$

/ Loop Ordering



```
for (m2=0; m2<M2; m2++) {  
    // IA buffer stores:  $M1 \times K$   
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K$   
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);  
        compute_matmul(*W, *IA, *OA,  
                        N2, N1, N0,  
                        M2, M1,  
                        K1, K0,  
                        m2, n2);  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0  
              , m2*M1:(m2+1)*M1]);  
    }  
}
```

/ Loop Ordering

Option 1: Loop_m2 -> Loop_n2

```
for (m2=0; m2<M2; m2++) {  
    // IA buffer stores: M1*K  
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores: N1*N0*K  
        mvin(W[0:K, n2*N1*N0:  
              (n2+1)*N1*N0]);  
        compute_matmul(*W, *IA, *OA,  
                        ...  
                        m2, n2);  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0  
               , m2*M1:(m2+1)*M1]);  
    }  
}
```

IA movement: $M * K$

W movement: $M2 * N * K$

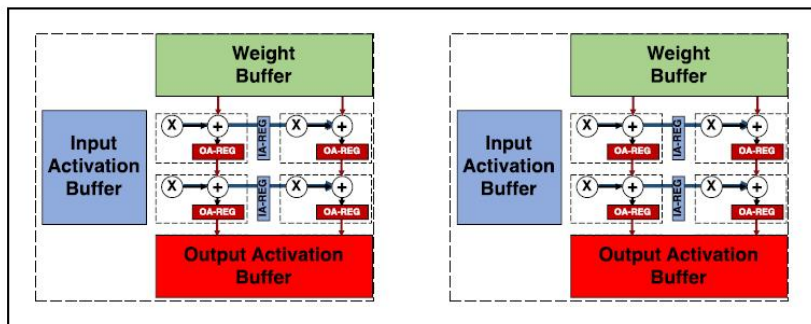
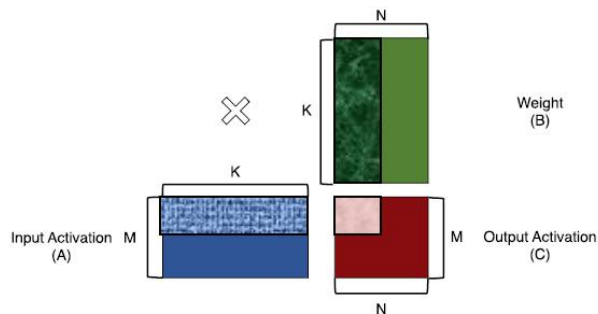
Option 2: Loop_n2 -> Loop_m2

```
for (n2=0; n2<N2; n2++) {  
    // W buffer stores: N1*N0*K  
    mvin(W[0:K, n2*N1*N0:  
          (n2+1)*N1*N0]);  
    for (m2=0; m2<M2; m2++) {  
        // IA buffer stores: M1*K  
        mvin(IA[m2*M1:(m2+1)*M1, 0:K]);  
        compute_matmul(*W, *IA, *OA,  
                        ...  
                        m2, n2);  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0  
               , m2*M1:(m2+1)*M1]);  
    }  
}
```

IA movement: $N2 * M * K$

W movement: $N * K$

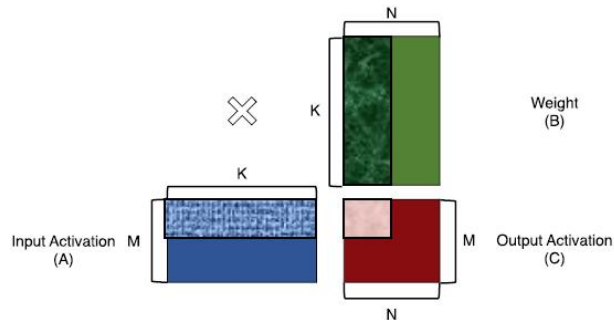
/Spatial Choice



```

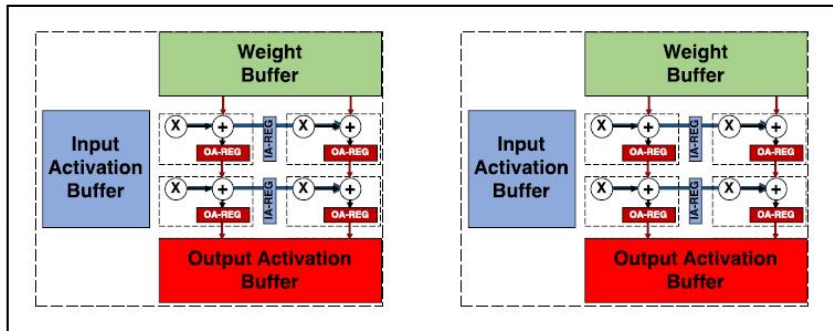
spatial_for (n2=0; n2<N2; n2++) {
    // Model Parallelism
    mvin(W[0:K, n2*N1*N0:
        (n2+1)*N1*N0]);
    for (m2=0; m2<M2; m2++) {
        mvin(IA[m2*M1: (m2+1)*M1, 0:K]);
        compute_matmul(*W, *IA, *OA,
            ...
            m2, n2);
        mvout(OA[n2*N1*N0: (n2+1)*N1*N0
            , m2*M1: (m2+1)*M1]);
    }
}
    
```

/Spatial Choice



```

spatial_for (m2=0; m2<M2; m2++) {
    // Data Parallelism
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);
    for (n2=0; n2<N2; n2++) {
        mvin(W[0:K, n2*N1*N0:
            (n2+1)*N1*N0]);
        compute_matmul(*W, *IA, *OA,
            ...
            m2, n2);
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0
            , m2*M1:(m2+1)*M1]);
    }
}
    
```



/Outlines

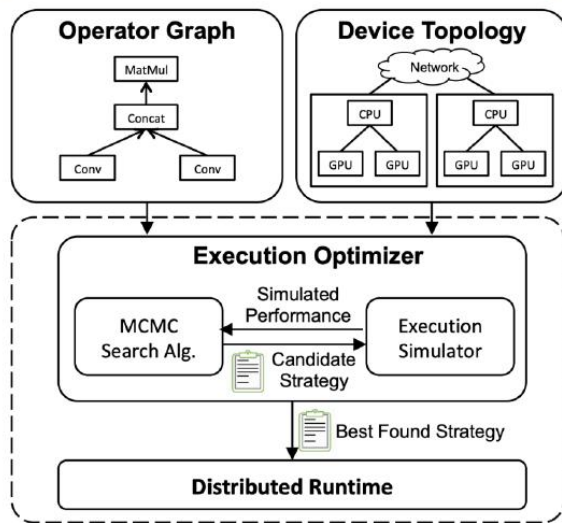
- **DNN Mapping Problem**
 - **Loop Nest**
 - **HW Constraints**
- **Mapping Space**
 - **Loop Ordering**
 - **Loop Bound**
 - **Spatial Choice**
- **Tuning**
- **Toolchain**

/ Navigating the Mapping Space

- DNN mapping problem can be viewed as an optimization problem.
- Given:
 - DNN dimensions (N, H, W, C, R, S, K, stride, padding)
 - Hardware specifications (dataflow, memory hierarchy)
- Find:
 - An optimal loop nest that minimizes latency and/or energy.
 - That defines both temporal and spatial execution order
- Approaches:
 - Exhaustive search
 - Random search
 - Learning-based algorithms

/ Example: FlexFlow, SysML 2018

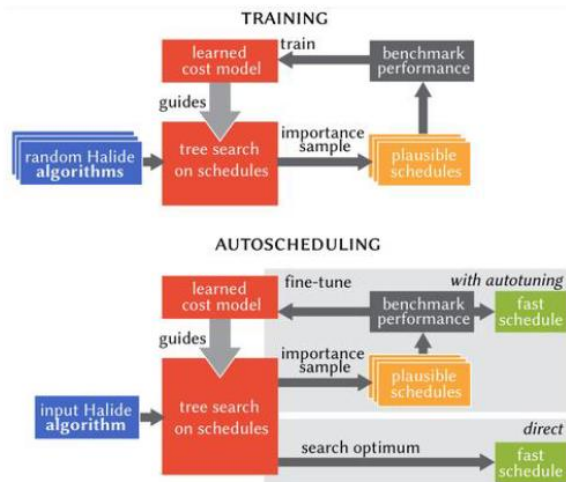
- “The optimizer uses a MCMC search algorithm to explore the space of possible parallelization strategies and iteratively proposes candidate strategies that are evaluated by an execution simulator.”



Beyond Data and Model Parallelism for
Deep Neural Networks, SysML 2018

/ Example: Halide, SIGGRAPH 2019

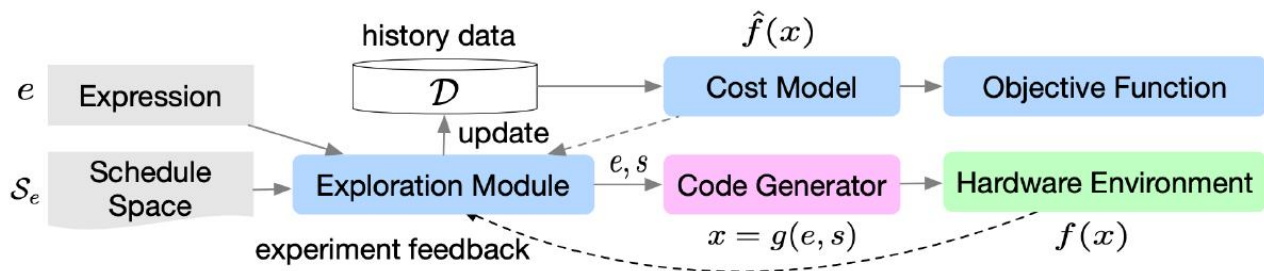
- “We generate schedules for Halide programs using tree search over the space of schedules guided by a learned cost model and optional autotuning. The cost model is trained by benchmarking thousands of randomly-generated Halide programs and schedules. The resulting code significantly outperforms prior work and human experts.”



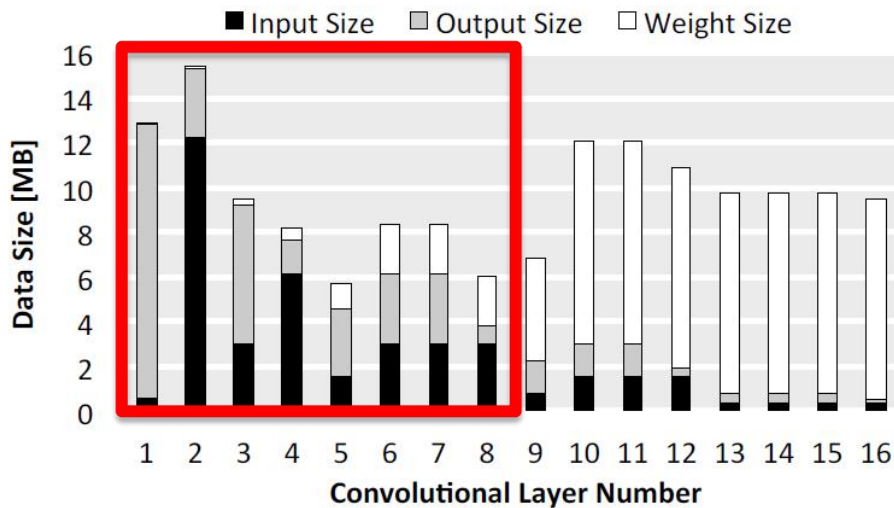
Learning to Optimize Halide with Tree Search and Random Programs, *SIGGRAPH 2019*

/ Example: TVM, NeurIPS 2018

- “We learn domain-specific statistical cost models to guide the search of tensor operator implementations over billions of possible program variants. We further accelerate the search using effective model transfer across workloads.”



/Motivation of Layer Fusion

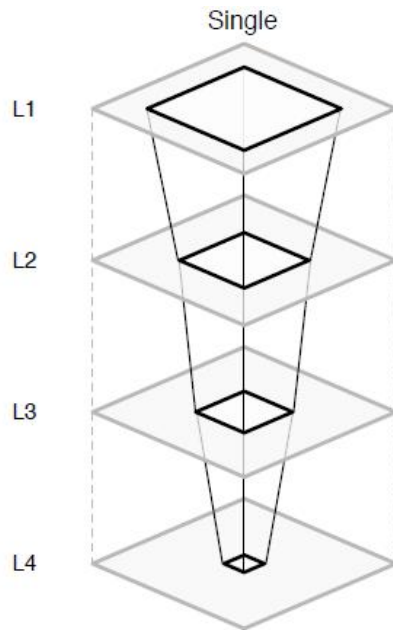
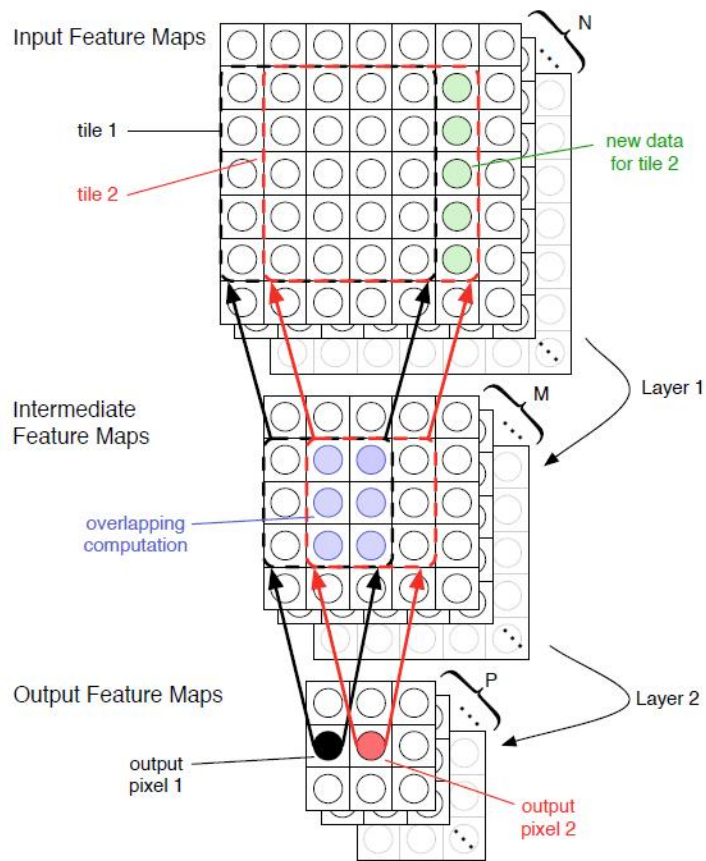


连续卷积层：前层输出=后层输入
传统方法：每层需要片下DRAM输入，层间独立。

如何跨层降低数据带宽？

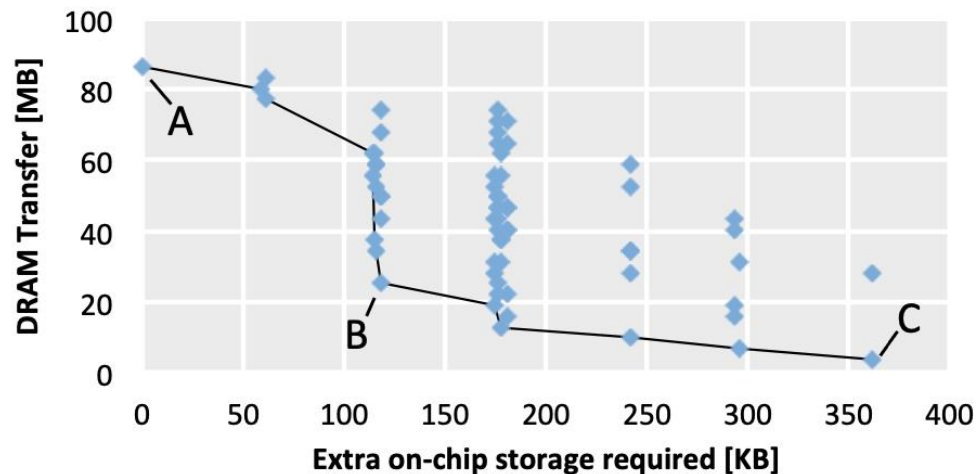
Fused-Layer CNN Accelerators, Micro 2016, Manoj. et.al.

/Layer Fusion Concept



- 多层数据同时缓存于片上
- **金字塔式输入特征图**
- 增量式瓦片 (tile) 数据
- 重叠计算保持片上数据

/Memory Access under Various Fusion Size



(b) VGGNet-E

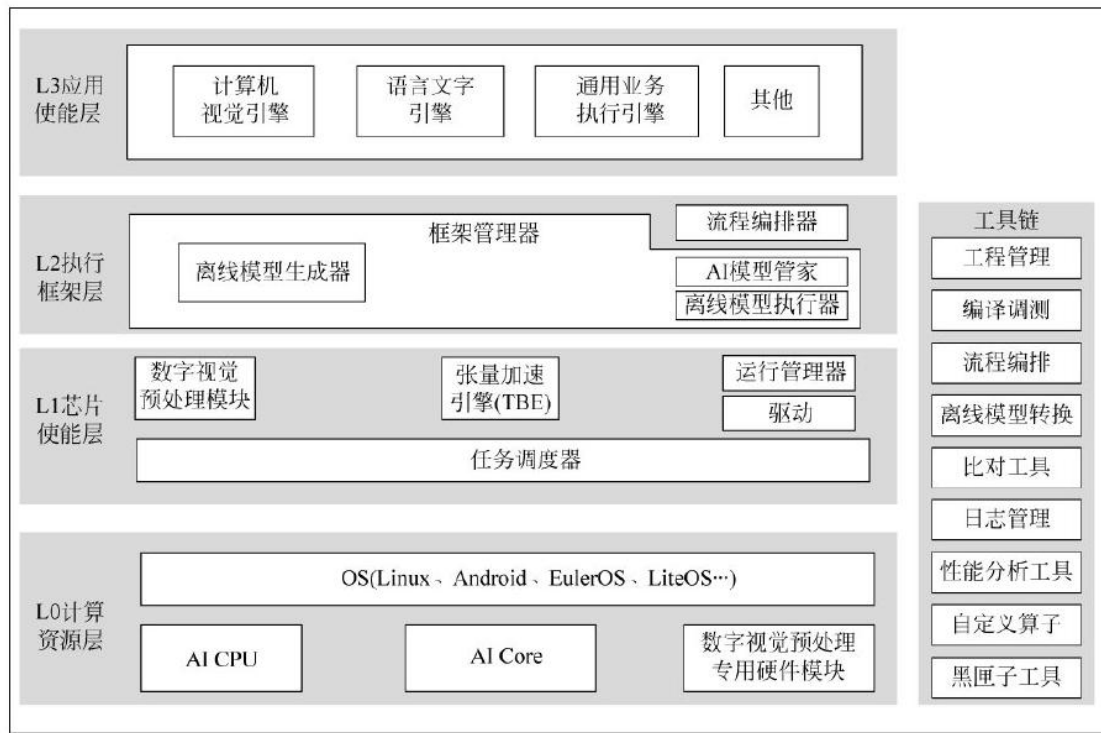
- 邻近原点数据点具有更好性能
- A: No fusion, 86MB DRAM
- C: All fusion, 3.6MB DRAM

	Fused-Layer	Baseline
MB transferred/input	3.64	77.14
Cycles $\times 10^3$	11,665	10,951
BRAMs	2,509	2,085
DSP48E1	2,987	2,880

/Outlines

- **DNN Mapping Problem**
 - **Loop Nest**
 - **HW Constraints**
- **Mapping Space**
 - **Loop Ordering**
 - **Loop Bound**
 - **Spatial Choice**
- **Tuning**
- **Toolchain**

软件栈（以华为昇腾为例）



四个层次，一个辅助工具链。

L3应用使能层：应用级封装，面向应用领域提供处理算法。

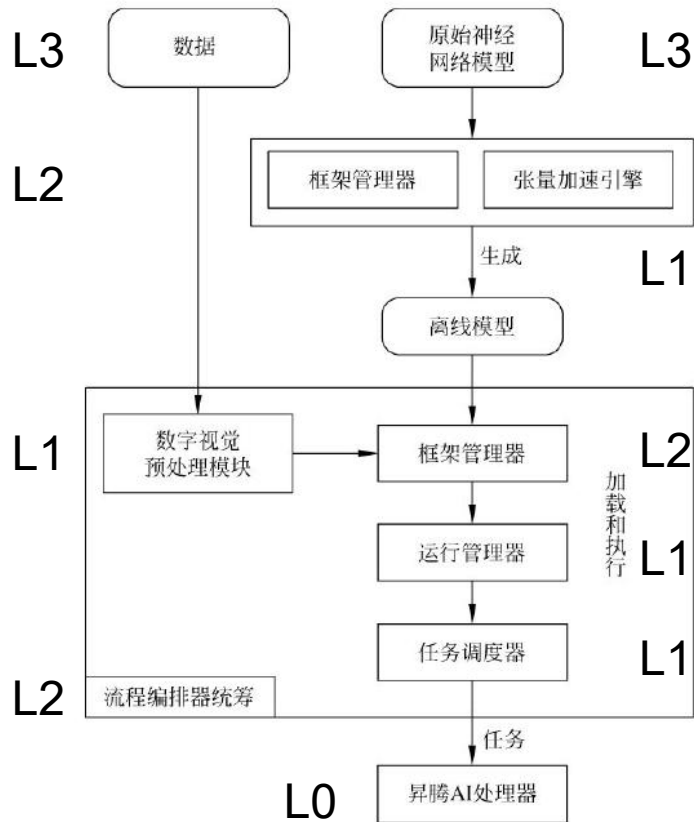
L2执行框架层：将算法进行深度学习框架的调用（框架管理器），离线深度学习模型生成（流程编排器）。

L1芯片使能层：给硬件通过加速库输出算子层面的任务，类似于生成机器码。

L0计算资源层：实际硬件。

工具链：开发平台。

神经网络软件流



实际的软件流并非严格单向，同时具有需要多层协同调度优化的需求。

L3应用使能层：应用级封装，面向应用领域提供处理算法。

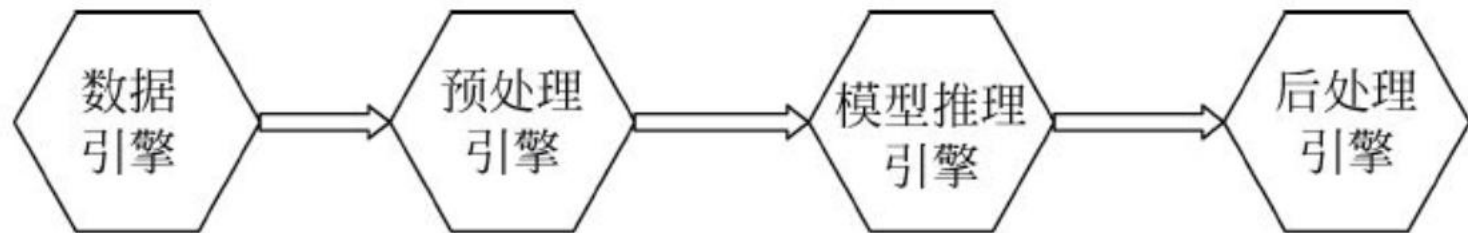
L2执行框架层：将算法进行深度学习框架的调用（框架管理器），以及离线深度学习模型生成（流程编排器）。

L1芯片使能层：给硬件通过加速库输出算子层面的任务，类似于生成机器码。

L0计算资源层：实际硬件。

工具链：开发平台。

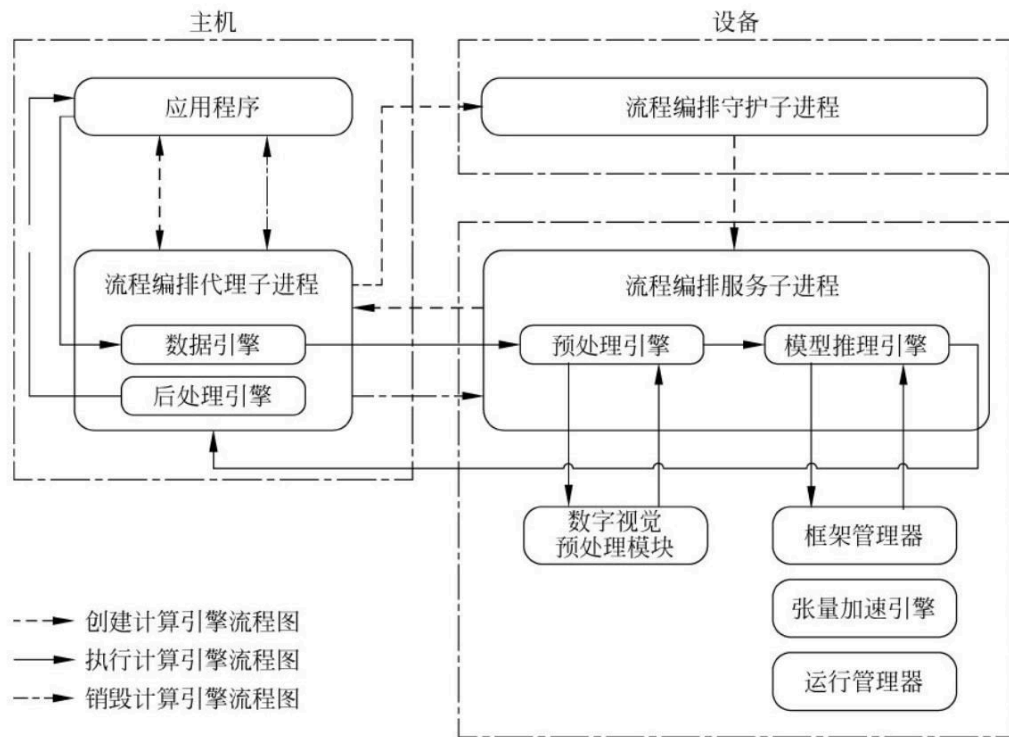
神经网络软件流



神经网络软件流



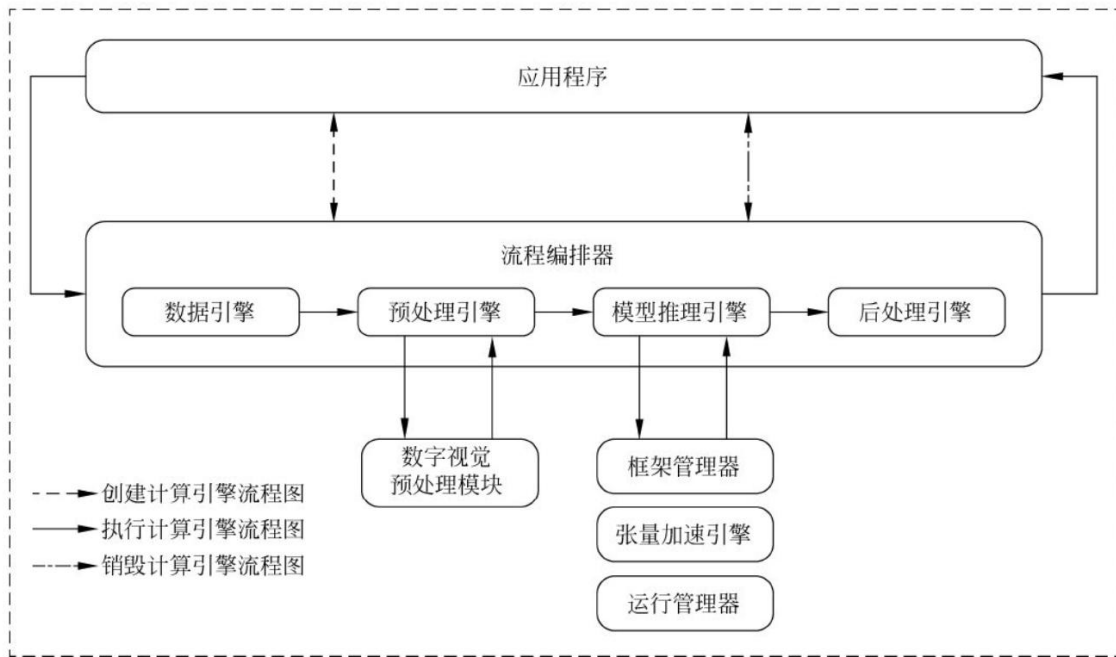
图 4-4 基于昇腾 AI 处理器的
PCIe 加速卡



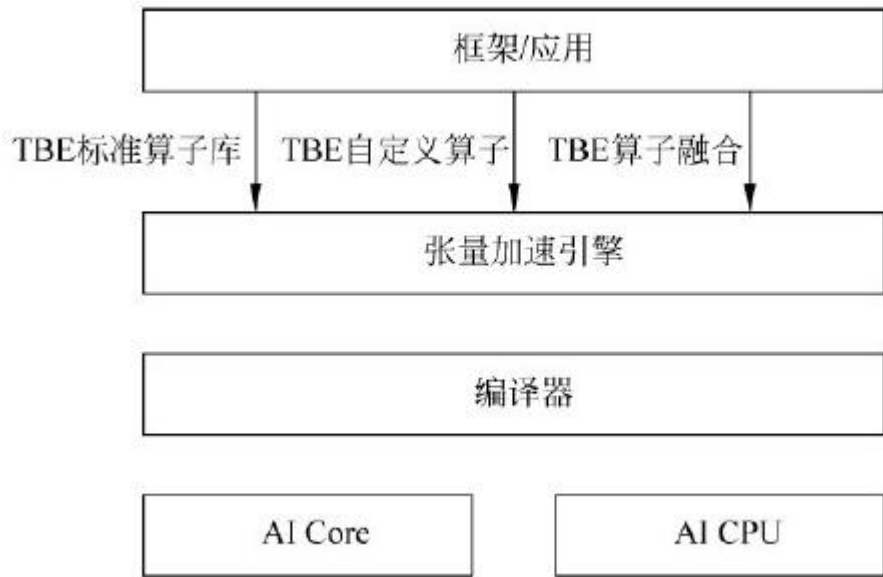
神经网络软件流



图 4-6 Atlas 200 DK



张量加速引擎

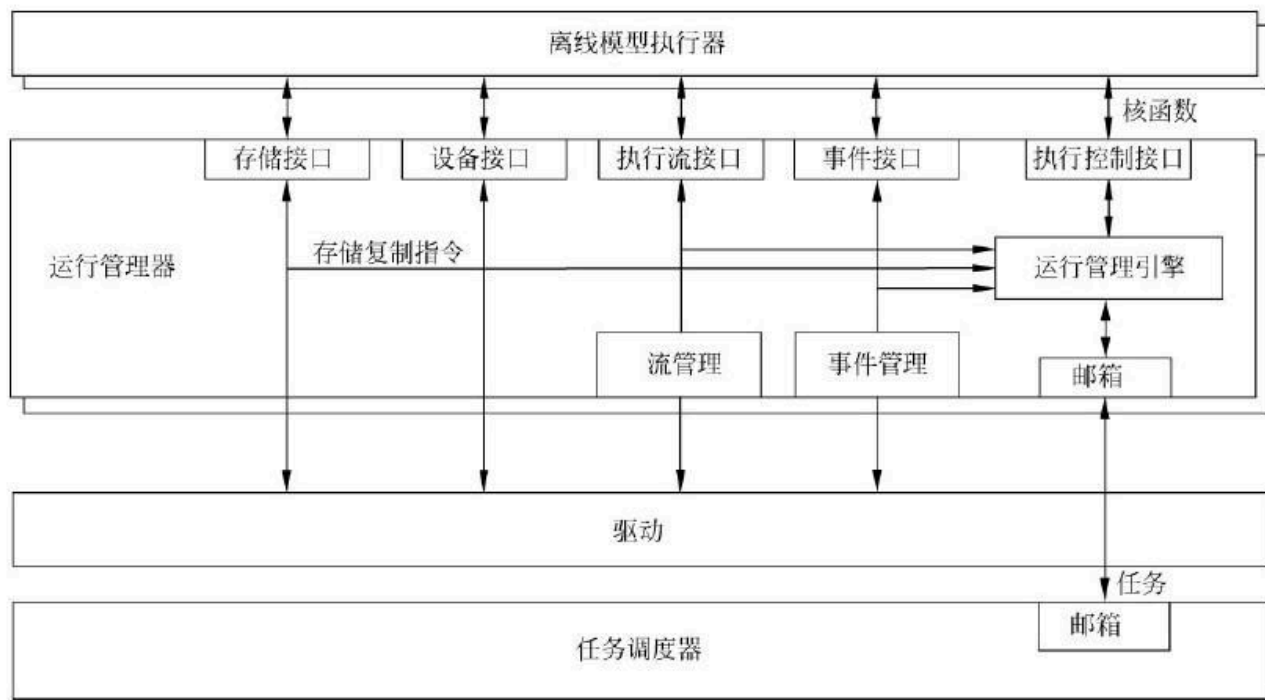


标准算子：卷积、ReLU等

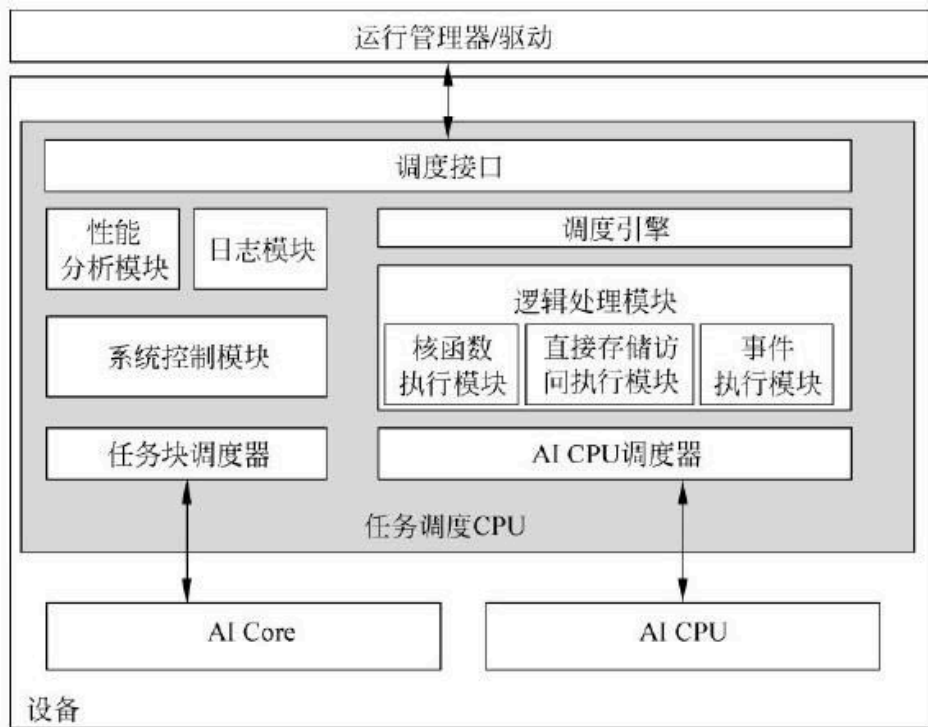
自定义算子：类似于CUDA C++编程，对新算子的高效硬件支持。

算子融合：cache多层缓存融合。

/ 运行管理器



任务调度器



调度流程

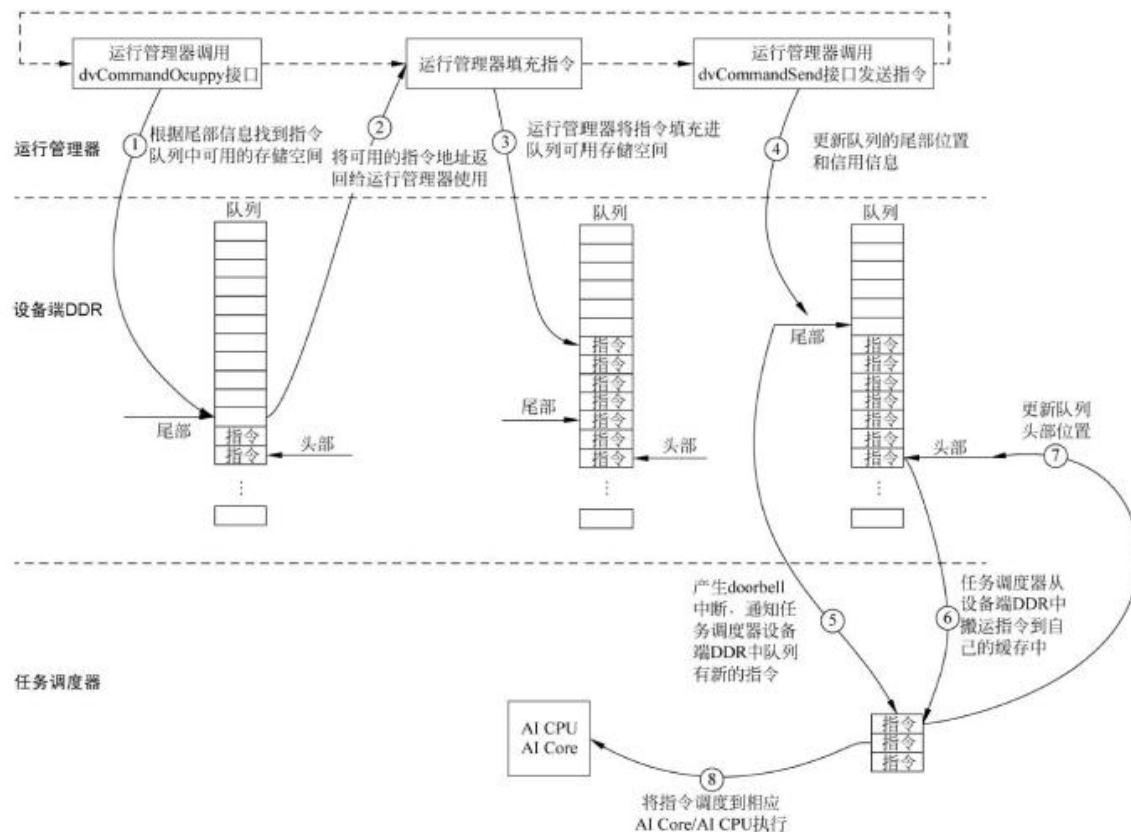
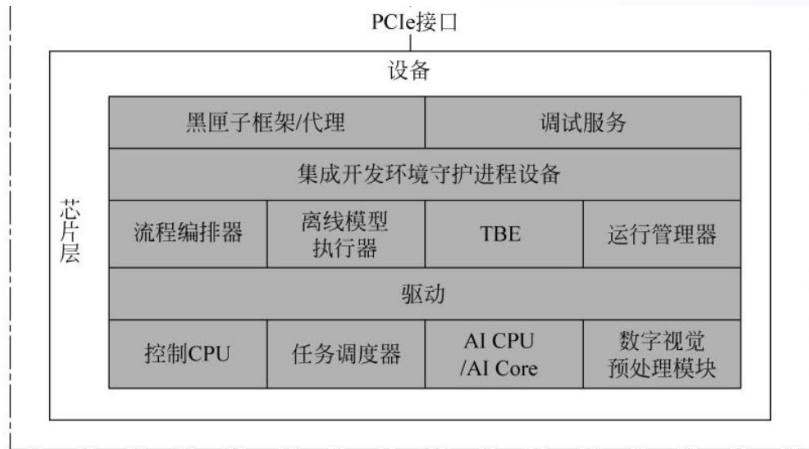
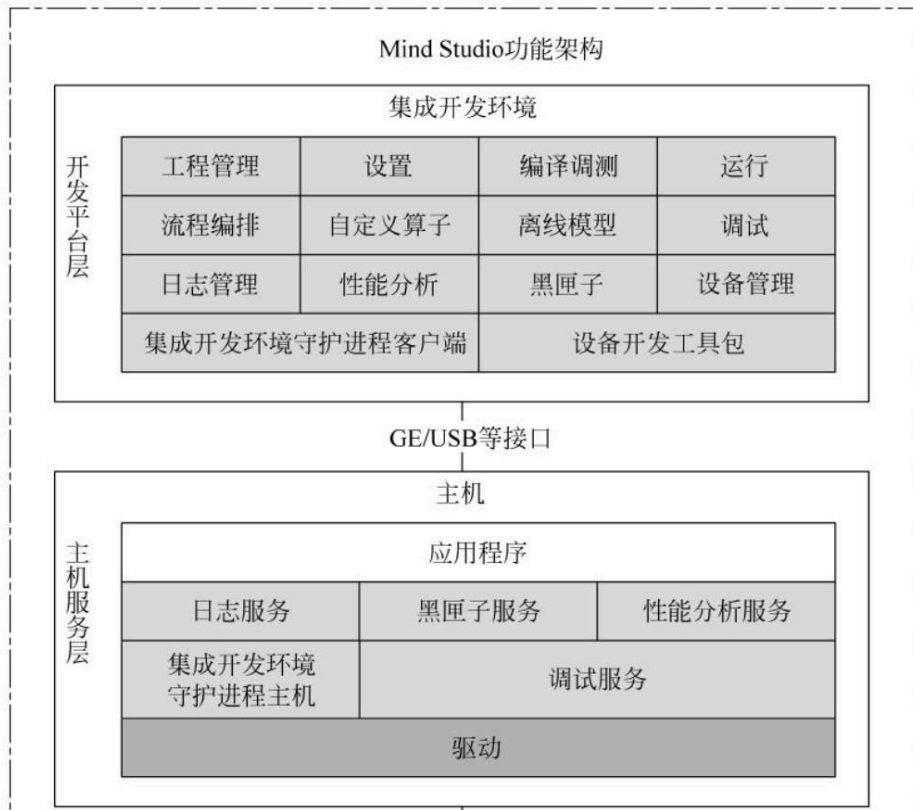


图 4-15 运行管理器和任务调度器协同运作流程图

工具链

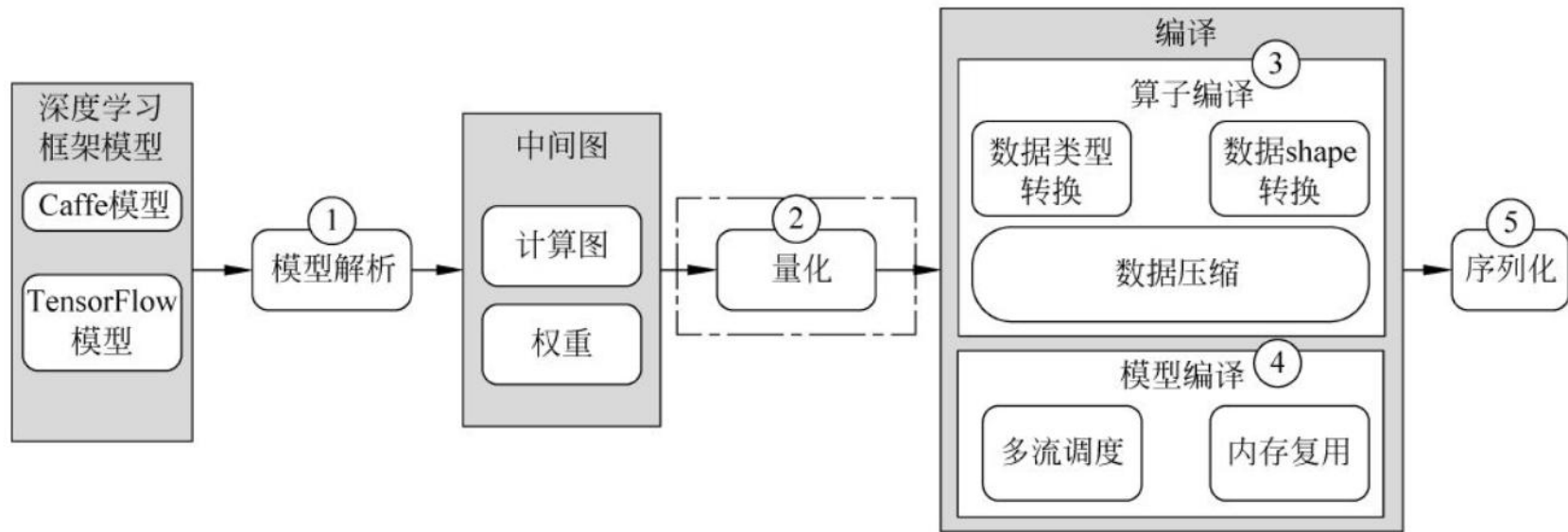


应用程序

Mind Studio
功能部件

基础软件和设备

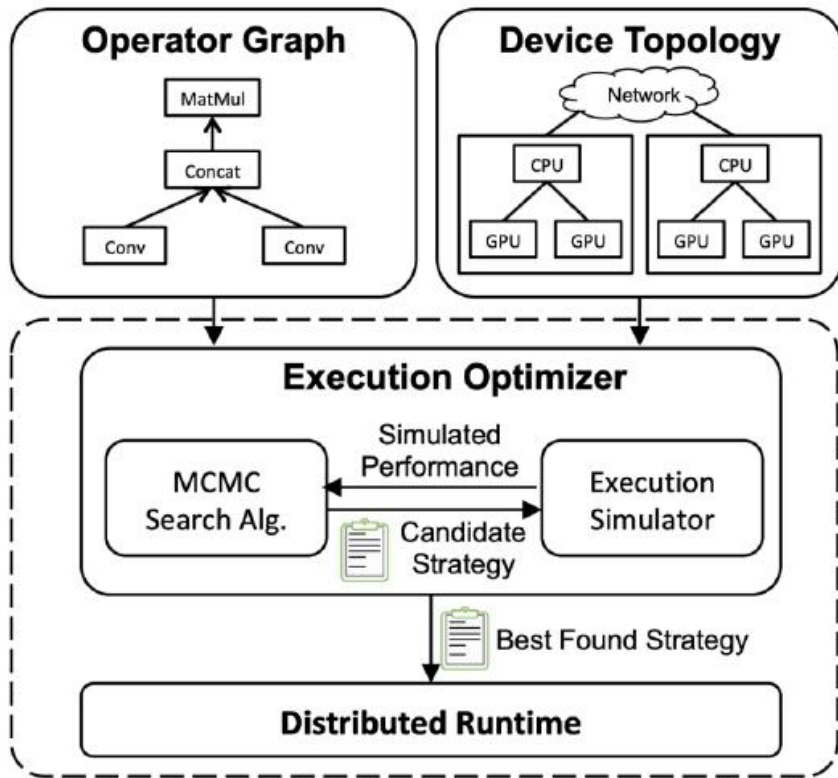
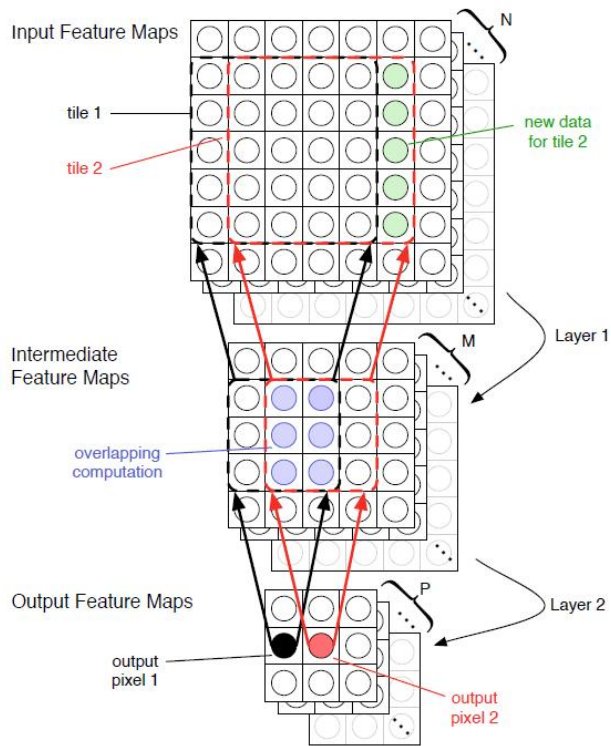
离线模型生成



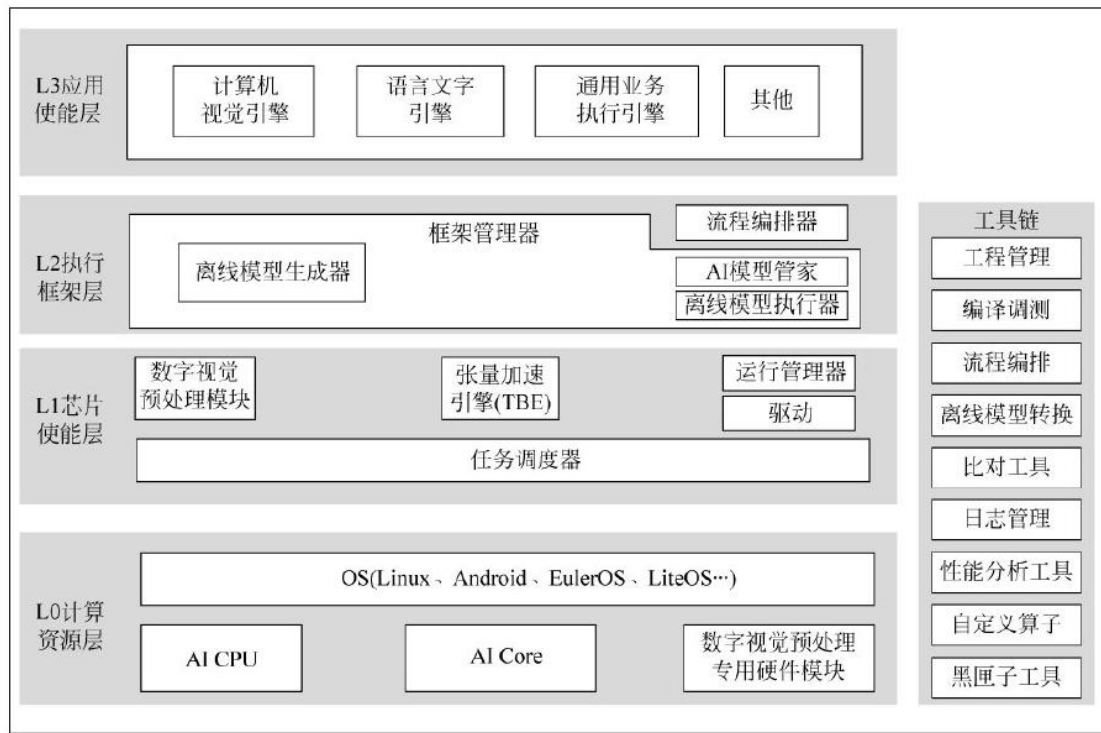
/Review

- Core computation in DNN
- Execution order of the core computation
- Hardware realization of the core computation
- This lecture: mapping DNNs to hardware
 - Temporal and Spatial Mapping based on hardware constraints.
 - Memory hierarchy
 - Parallelism
 - Tile the loops to improve reuse and parallelism
 - Loop ordering
 - Loop bounds
 - Spatial choices
 - Navigate the large mapping space

/Review: Tuning



Review



四个层次，一个辅助工具链。

L3应用使能层：应用级封装，面向应用领域提供处理算法。

L2执行框架层：将算法进行深度学习框架的调用（框架管理器），离线深度学习模型生成（流程编排器）。

L1芯片使能层：给硬件通过加速库输出算子层面的任务，类似于生成机器码。

L0计算资源层：实际硬件。

工具链：开发平台。